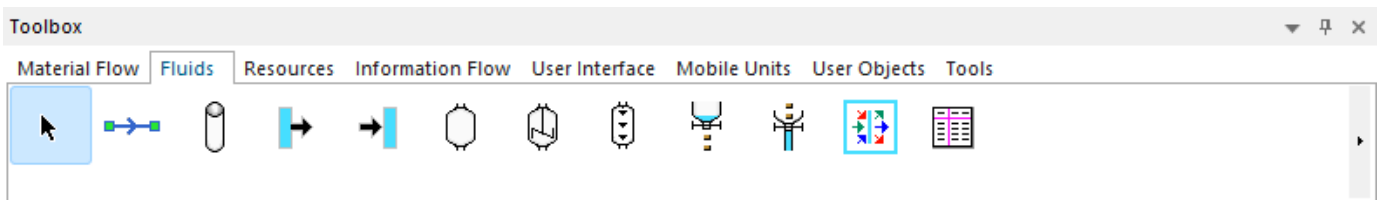


Simulating Free-Flowing Materials and Fluids

Simulate Free-flowing Materials and Fluids

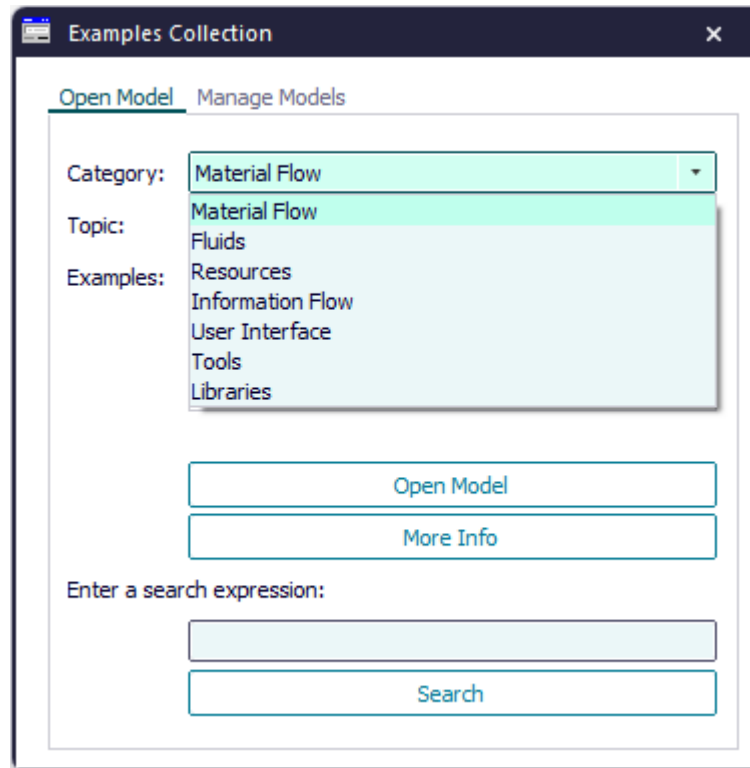
Plant Simulation provides the **Fluid Objects** for simulating free-flowing materials. These materials can be in liquid, gaseous, or pourable form. The fluid objects are suited for the food and beverages and for the pharmaceutical industries.



In our examples we will demonstrate how to model the following scenarios:

- **Model the Production of Chocolate Bars**
- **Continuously Mix Juice and Water**
- **Portion and Deportment Materials**
- **Produce Fluids in a Fixed Sequence**

You can also check out the extended model containing additional stations and controls in the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection** and click **Open Model**.



Back to [Modeling the Material Flow](#)

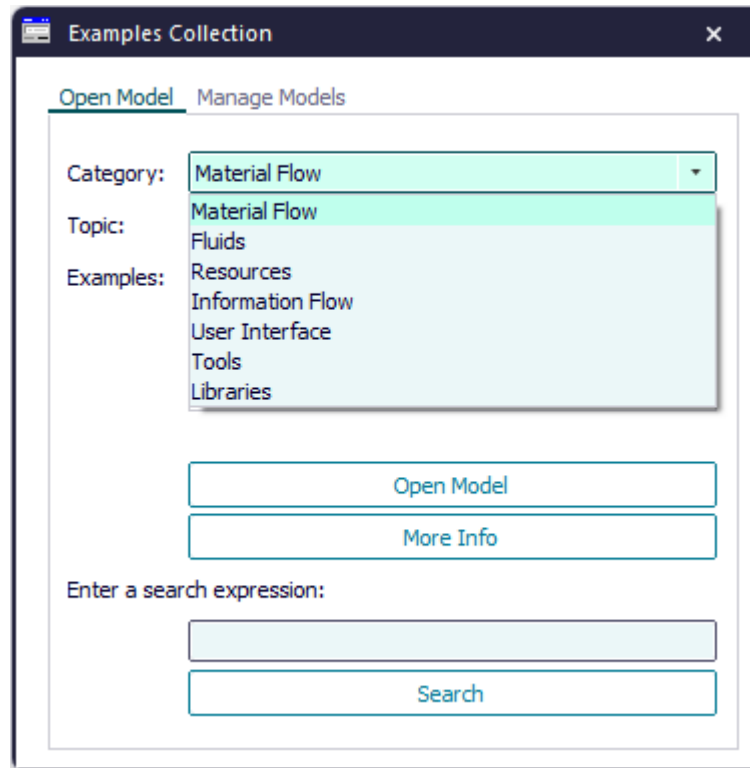
Modeling the Production of Chocolate Bars

Model the Production of Chocolate Bars

In our example we will model a simple production line for making chocolate bars.

The *FluidSources* ➔ provide the ingredients **milk**, **sugar**, and **cocoa**. These then flow through *Pipes* ⚙️ to their individual *Tanks* 🛢️, where they are temporarily stored before flowing into the first *Mixer* 🌀. It mixes them together to our first intermediate product, the **base chocolate**. The **base chocolate** is then pumped to the next *Mixer*, which stirs the mixture to the desired consistency. The resulting **chocolate mass** flows on to the final *Mixer*, which blends in the refining material **chili** that is fed in from its own *FluidSource* ➔. The resulting **chili chocolate** flows to the *Portioner* 🍷, which pours the individual chocolate bars. These are then moved on a *Conveyor* to the *Drain*, which removes the chocolate bars from the plant.

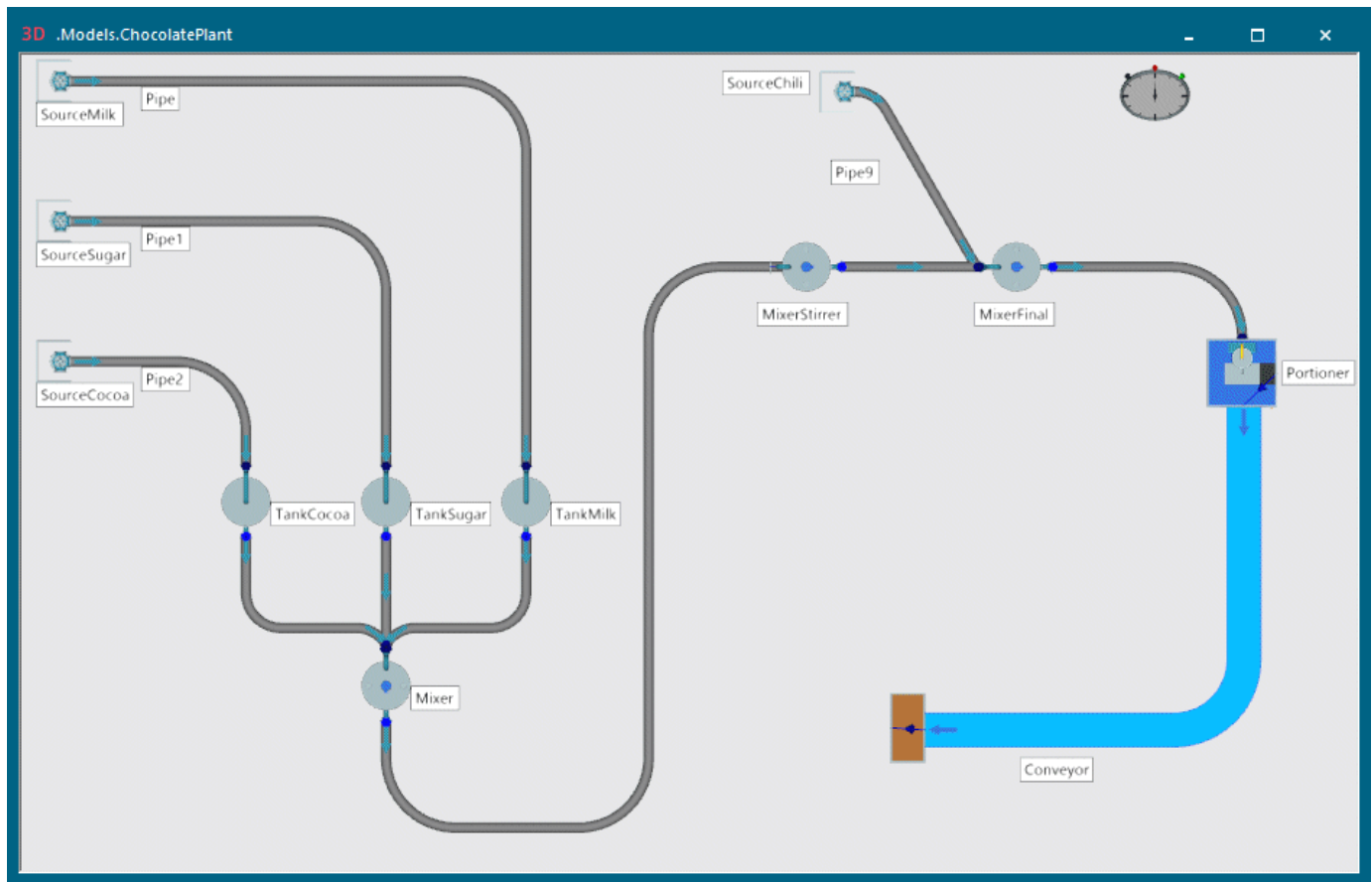
You can also consult an expanded model with additional stations and controls in the example models collection: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection** and click **Open Model**.



In our example we will demonstrate how to:

- **Configure the Recipe in the MaterialsTable**
- **Configure the FluidSources Providing the Materials**
- **Configure the Tanks Storing the Materials**
- **Configure the Mixers Transmuting the Materials**
- **Configure the Source Providing the Refining Material**
- **Configure the Portioner Pouring the Chocolate Bars**
- **Configure the Conveyor and Run the Simulation**
- **Change the Shape of the Pipes**

The finished simulation model looks like this:



Back to [Simulate Free-flowing Materials and Fluids](#)

Compare [Portion and Deportion Materials](#)

Configure the Recipe in the MaterialsTable

Before we start building our simulation model, we define the **ingredients** and the products in the object *MaterialsTable*.

These are then going to be used by the *FluidSource*  and by the *Mixer* .

You can enter the materials and the information pertaining to them in the order in which they occur to you. You are not bound to the order in which the materials are used within the production process. As long as the materials are contained in the **MaterialsTable** , the object, which needs them, will find them.

In most cases you can do that in the *MaterialsTable* in the *class library*, meaning that you do not have to insert an instance of it into the *Frame* that contains your simulation model.

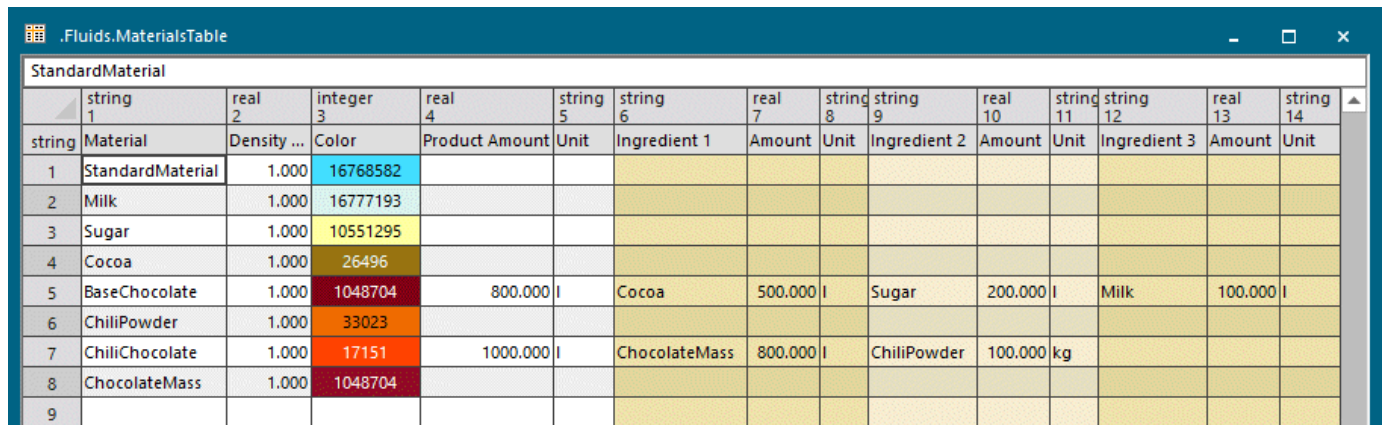
If your simulation model contains a number of *Frames* which require a vast number of different materials, we recommend to insert different *MaterialsTables* into the *Frames*. A single *MaterialsTable* might become too large and unwieldy to handle.

We entered the **base products**, their **density**, their **color**, and the **amount** which we want to produce. We also entered the **intermediate products** and the **final product**, their **density**, their **color**, the **amount** which we want to produce, and the **ingredients** these products require.

Note

When you change the name of materials or of ingredients at a later point in time in the *MaterialsTable*, you also have to manually change this name in the text box **Material** of the *FluidSource* and/or the text box **Product** of the *Mixer*. *Plant Simulation* does not automatically do that for you.

Our completed *MaterialsTable* looks like this.



	string 1	real 2	integer 3	real 4	string 5	string 6	real 7	string 8	string 9	real 10	string 11	string 12	real 13	string 14
	Material	Density ...	Color	Product Amount	Unit	Ingredient 1	Amount	Unit	Ingredient 2	Amount	Unit	Ingredient 3	Amount	Unit
1	StandardMaterial	1.000	16768582											
2	Milk	1.000	16777193											
3	Sugar	1.000	10551295											
4	Cocoa	1.000	26496											
5	BaseChocolate	1.000	1048704	800.000	l	Cocoa	500.000	l	Sugar	200.000	l	Milk	100.000	l
6	ChiliPowder	1.000	33023											
7	ChiliChocolate	1.000	17151	1000.000	l	ChocolateMass	800.000	l	ChiliPowder	100.000	kg			
8	ChocolateMass	1.000	1048704											
9														

Compare Add Additional Ingredients to the MaterialsTable

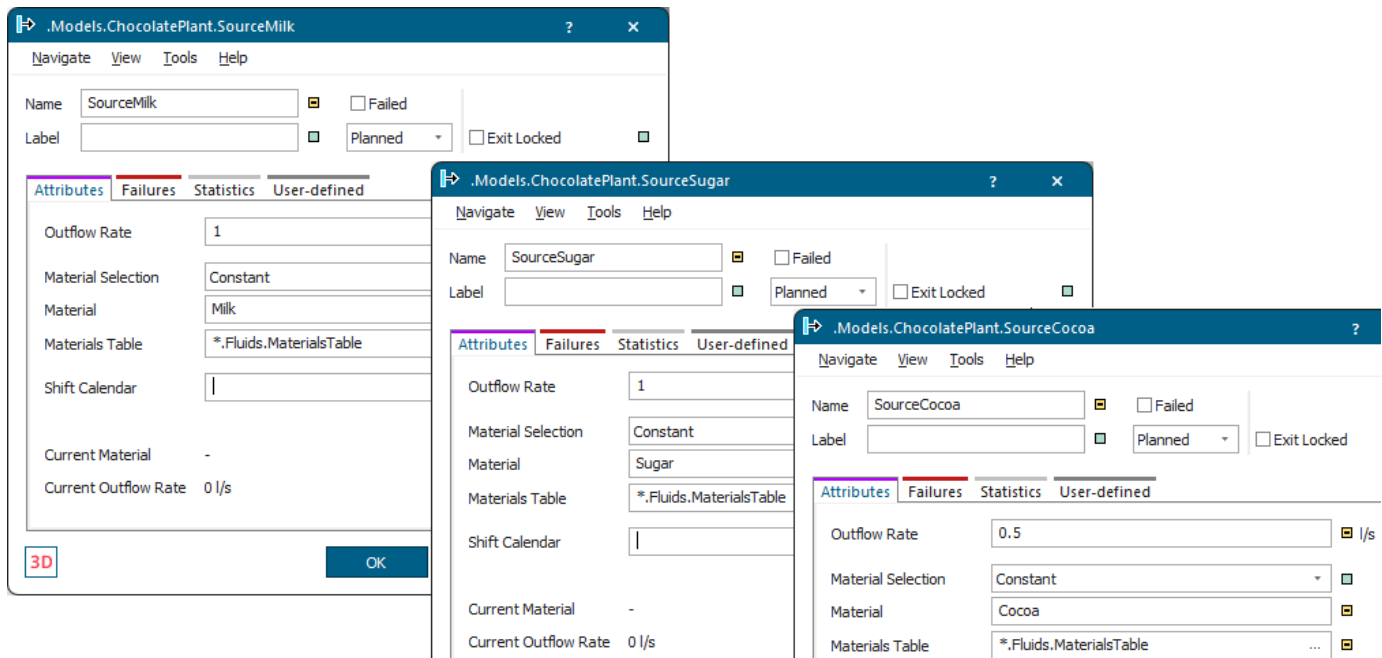
Back to [Model the Production of Chocolate Bars](#)

Configure the FluidSources Providing the Materials

After filling in the *MaterialsTable*, we will configure the objects of type *FluidSource*, which introduce the base materials into the plant.

We inserted a **FluidSource**  each for the base materials **milk**, **sugar**, and **cocoa**. We then typed the name of material into the text box **Material**, namely **Milk**, **Sugar**, and **Cocoa**. We did not change any other settings.

Configure the Tanks Storing the Materials



Back to Model the Production of Chocolate Bars

Configure the Tanks Storing the Materials

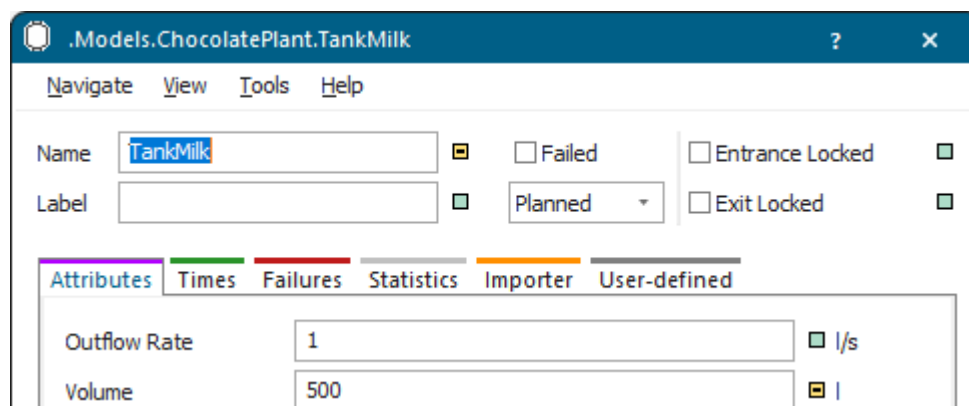
The *Tanks* temporarily store the base materials before they are mixed together in the *Mixer*.

Next, we inserted a [Tank](#) → each for the base materials **milk**, **sugar**, and **cocoa**.

We entered:

- The **outflow rate** for each of the materials into the text box **Outflow Rate** of the respective object.
- The **volume** for each of the materials into the text box **Volume** of the respective object.

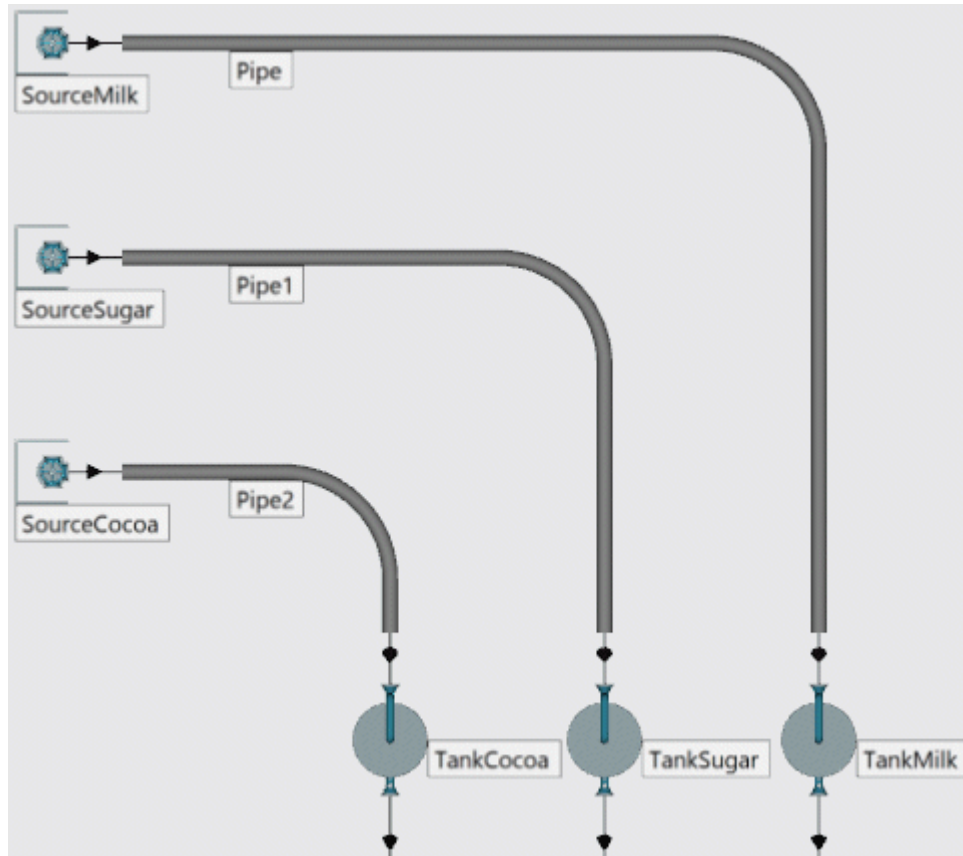
We did not change any other settings.



Next, we inserted *Pipes* between the *FluidSources* and the respective *Tanks*. We then connected the three *Pipes* with the *FluidSources* and the *Tanks* with *Connectors*.

Note

You have to connect the individual fluid objects with *Connectors*. You cannot connect them with *Pipes* only.



Back to [Model the Production of Chocolate Bars](#)

Configure the Mixers Transmuting the Materials

Then we inserted three objects of type *Mixer*, which transmute the **base materials** and the **intermediate product** to create the **end product**, namely the **chili chocolate bars**.

We entered:

- The **outflow rate** of each of the products into the text box **Outflow Rate** of the respective object.
- The **volume** of each of the products into the text box **Volume** of the respective object.
- The name of each of the **products** into the text box **Product** of the respective object.

- A **processing time** for each of the **Mixers**  on the tab **Times**.

We did not change any other settings.

.Models.ChocolatePlant.Mixer

Navigate View Tools Help

Name: ☐ Failed ☐ Entrance Locked ☐

Label: ☐ Planned ☐ Exit Locked ☐

Attributes Times Failures Controls Statistics Importer User-defined

Outflow Rate: ☐ l/s

Volume: ☐ l

Product: ☐

Product Amount: ☐ l

Materials Table: ... ☐

Current Fill Level: 0.00% Current Inflow Rate: 0 l/s

Current Amount: 0 l Current Outflow Rate: 0 l/s

3D OK Cancel Apply

.Models.ChocolatePlant.MixerStirrer

Navigate View Tools Help

Name: ☐ Failed ☐ Entrance Locked ☐

Label: ☐ Planned ☐ Exit Locked ☐

Attributes Times Failures Controls Statistics Importer User-defined

Outflow Rate: ☐ l/s

Volume: ☐ l

Product: ☐

Product Amount: ☐ l

Materials Table: ... ☐

Current Fill Level: 0.00% Current Inflow Rate: 0 l/s

Current Amount: 0 l Current Outflow Rate: 0 l/s

3D OK Cancel Apply

.Models.ChocolatePlant.MixerFinal

Navigate View Tools Help

Name: ☐ Failed ☐ Entrance Locked ☐

Label: ☐ Planned ☐ Exit Locked ☐

Attributes Times Failures Controls Statistics Importer User-defined

Outflow Rate: ☐ l/s

Volume: ☐ l

Product: ☐

Product Amount: ☐ l

Materials Table: ... ☐

Current Fill Level: 0.00% Current Inflow Rate: 0 l/s

Current Amount: 0 l Current Outflow Rate: 0 l/s

3D OK Cancel Apply

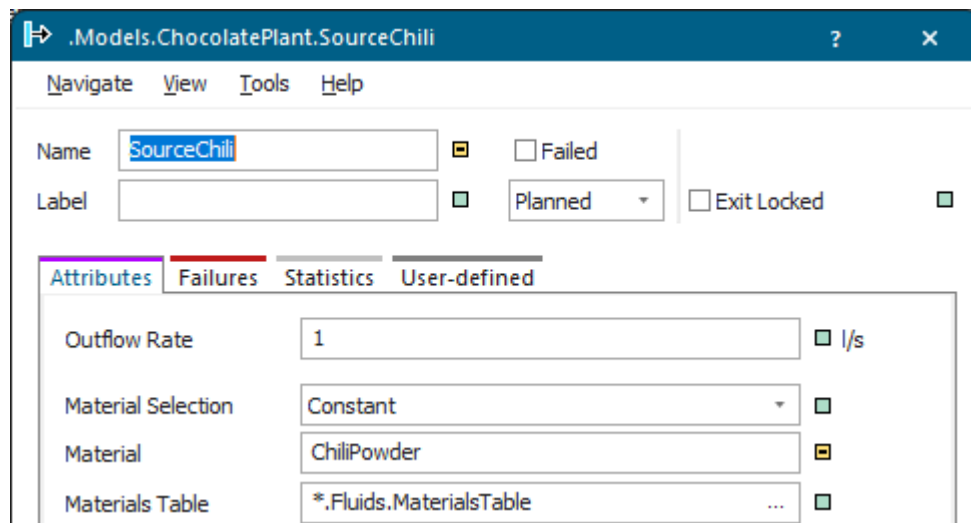
We then connected the *Mixers* with *Pipes* and with *Connectors*.

Back to Model the Production of Chocolate Bars

Configure the Source Providing the Refining Material

As we want to produce chili chocolate bars, we have to add the chili powder to our chocolate mixture in the last *Mixer* in the production line.

To do so, we insert a *FluidSource* and name it *SourceChili*. We then typed the name of material into the text box **Material**, namely *ChiliPowder*. We did not change any other settings.



We then connected the *SourceChili* with a *Pipe* and a *Connector* with the *MixerFinal*.

Back to Model the Production of Chocolate Bars

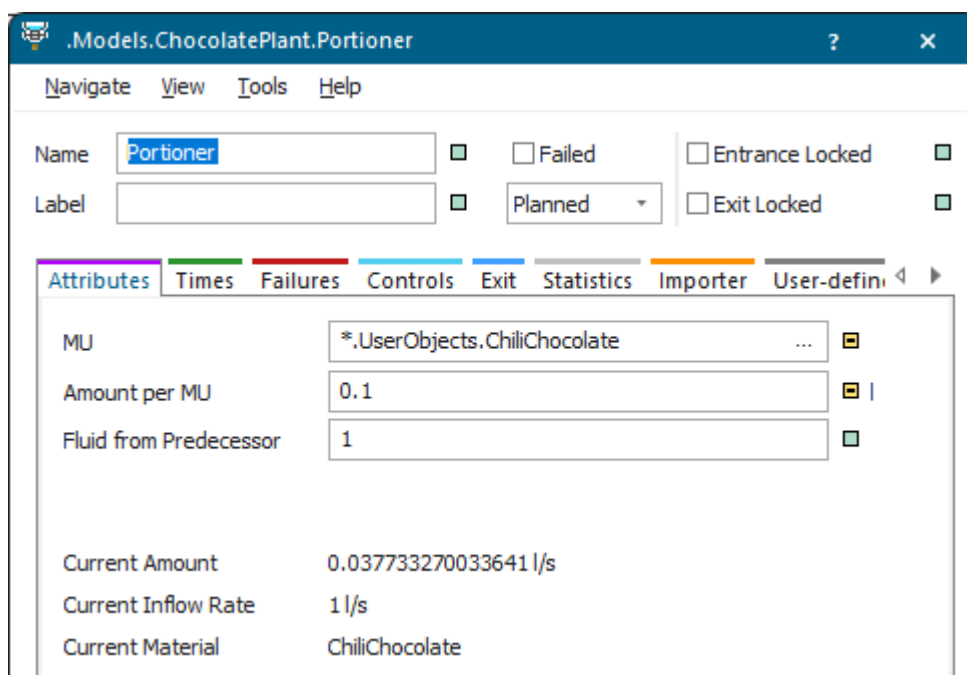
Configure the Portioner Pouring the Chocolate Bars

The last fluid object we use in our model is the *Portioner*. It pours the individual chocolate bars.

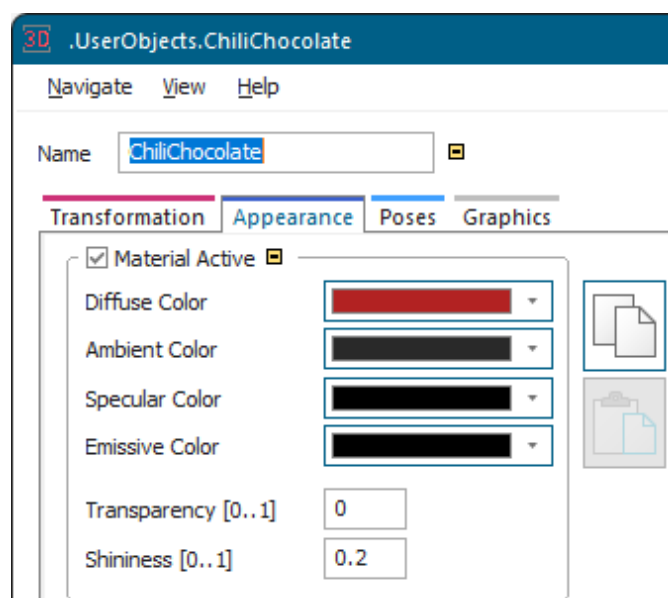
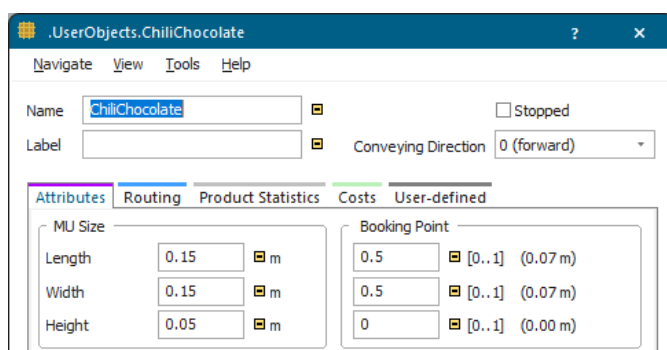
We entered:

- We selected the **MU** that the **Portioner** is to create from the products, namely the **ChiliChocolate**.
- We entered the **Amount per MU** of the product. In our example 0.1 liters of the chili chocolate is used to make a single chocolate bar.

We did not change any other settings.



To represent our chocolate bar in our model, we created a new *MU* class in the *Class Library* and named it *ChiliChocolate*. We used these settings:

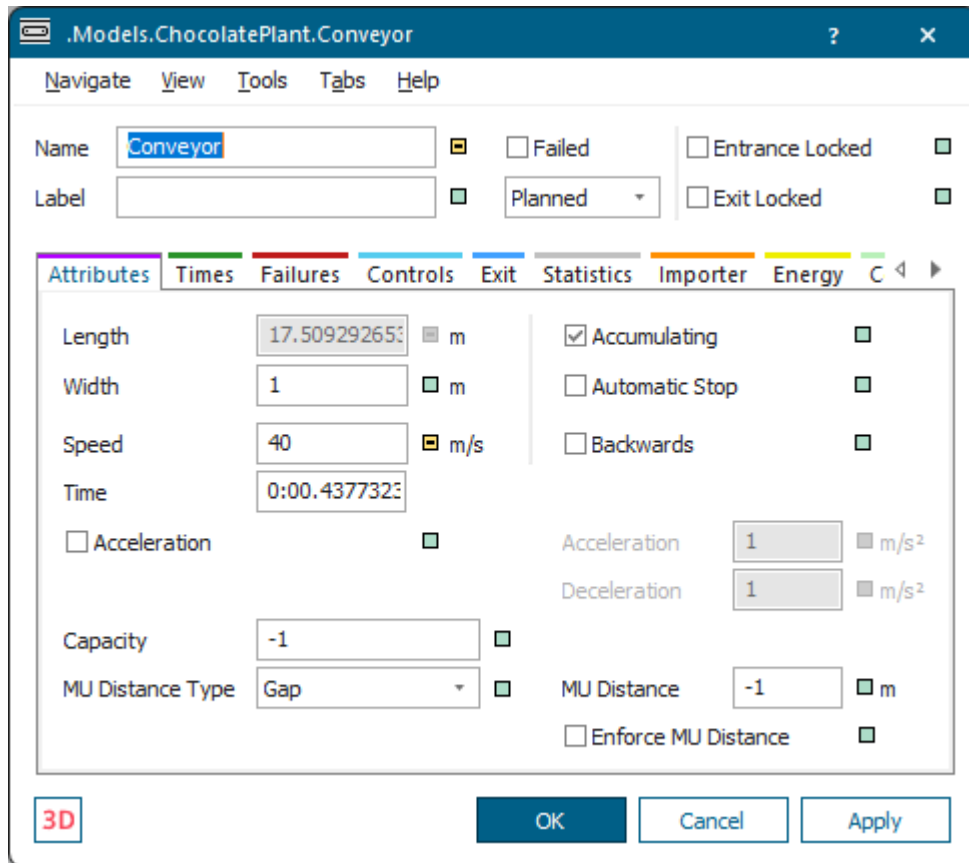


Back to Model the Production of Chocolate Bars

Configure the Conveyor and Run the Simulation

Finally, we inserted a *Conveyor*, which transports the chocolate bars from the *Portioner* to the *Drain*. The *Drain* then removes them from the plant.

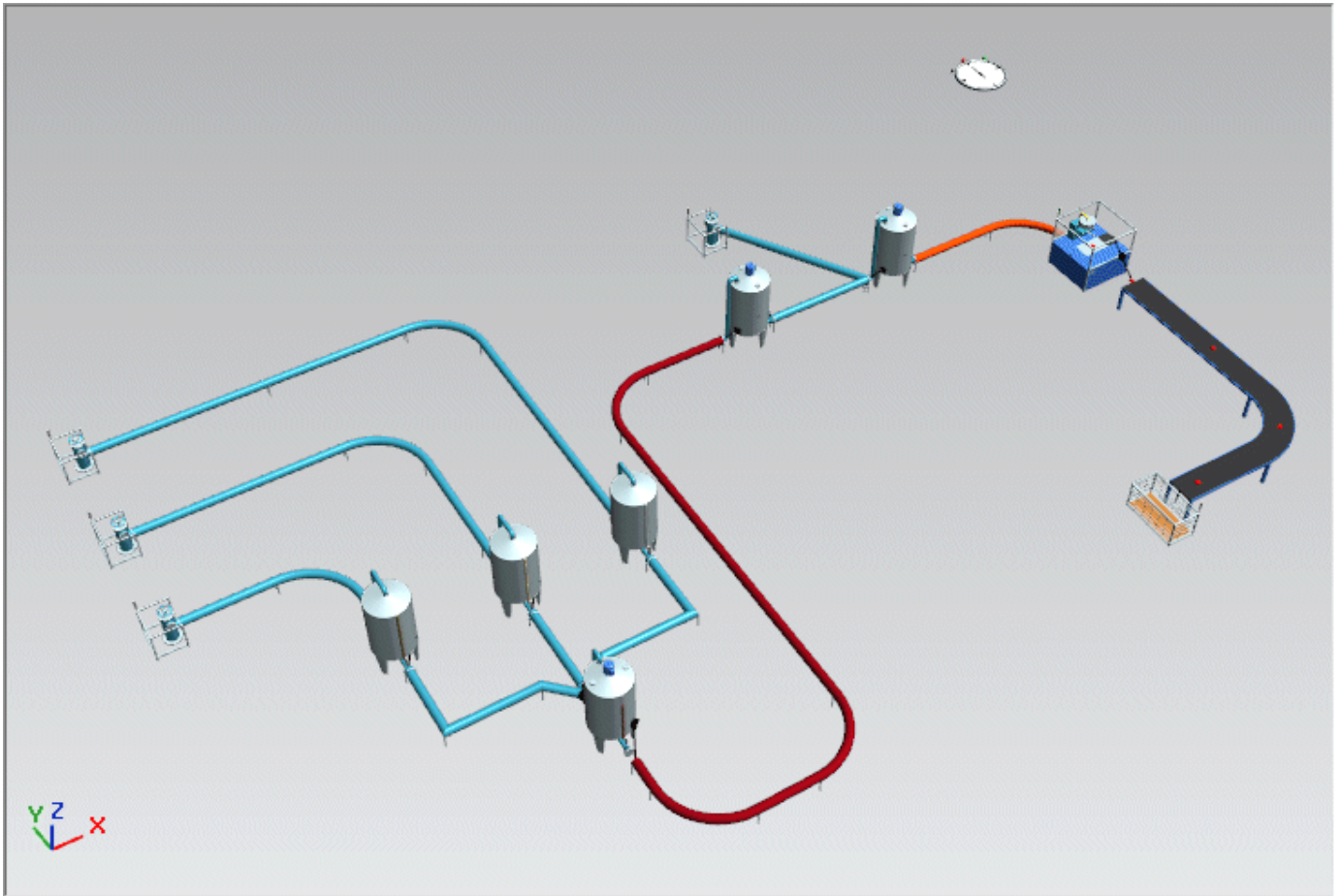
To be able to see the individual chocolate bars on the conveyor, we set the **Speed** of the *Conveyor* to 40 meters per second.



Then we inserted a *Drain*, which we used with its default settings and ran the simulation.

You'll notice that the *Tanks*, the *Mixers*, and the *Pipes* show the color of the material which we set in the *MaterialsTable* above. The chili chocolate bar, i.e., the *MU*, shows the color which we set for it on the tab **Graphics**.

As you see *Plant Simulation* displays the default graphic of the part with the default size for our chocolate bar. This is not the way we want it. To display the chocolate bar, we right-clicked the class of our *MU* named **ChiliChocolate** in the *Class Library* and selected **Edit 3D Properties**. We then clicked the tab **Transformation** and activated **Scale Automatically**. The chocolate bar then looks like this:

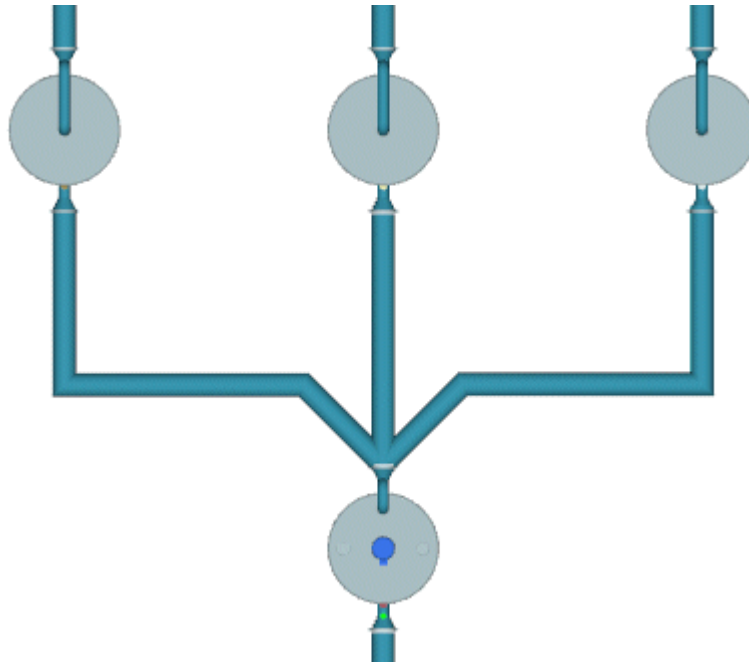


Back to [Model the Production of Chocolate Bars](#)

Change the Shape of the Pipes

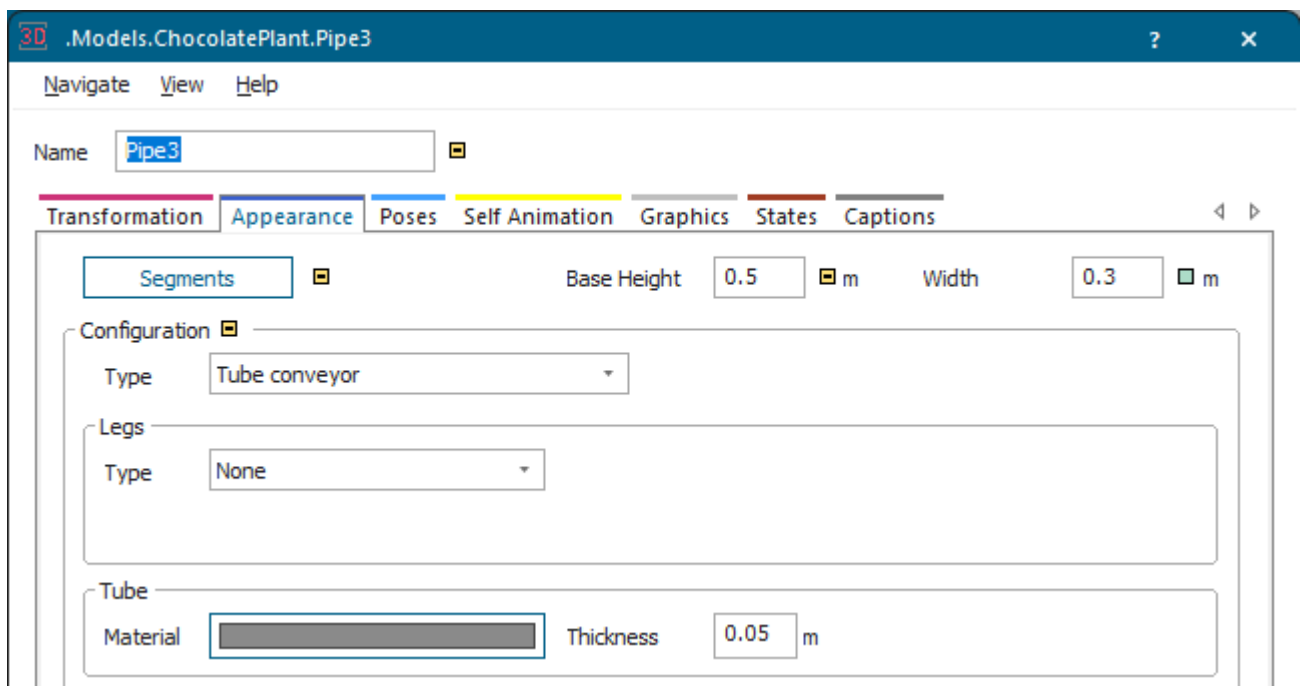
If you did not insert the pipes with curves initially, you can change the shape of three *Pipes* so that they match the real-life *Pipes*.

- In our example, the right angles connecting the *TankCocoa* and the *TankMilk* with the *Mixer* obstruct the free flow of these materials.



Instead of inserting the *Pipes* anew containing curves, we edit their **Segments** [context menu] tables.

- Click *Pipe3* once and press the **spacebar**. Click **Segments**.



- Click in the last row of the table and select **Append Row**.

We entered these values to bend the *Pipe*.

Segments – .Models.ChocolatePlant.Pipe3

I.	Tangential angle [°]	Length [m]	Curve angle [°]	Radius [m]	Vertical curve	ΔZ [m]	X [m]	Y [m]	Z [m]	ΔL [m]
1						0	-7.000	-2.600	0.500	0.000
2	90	1.7				0	-7.000	-4.300	0.500	1.700
3	0		-90	1	☐	0	-6.000	-5.300	0.500	3.271
4	0	2.15				0	-3.850	-5.300	0.500	5.421
5	0		65	1	☐	0	-2.944	-5.877	0.500	6.555

OK Cancel Apply

Note

To get an idea about the values you have to enter, you can insert a *Pipe* with the approximate shape of the existing *Pipe*, experiment with these values, and then delete it again.

- Repeat this for *Pipe5*. We entered these values.

Segments – .Models.ChocolatePlant.Pipe5

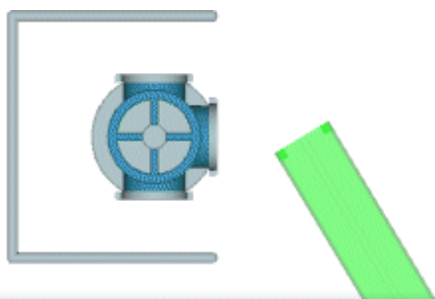
I.	Tangential angle [°]	Length [m]	Curve angle [°]	Radius [m]	Vertical curve	ΔZ [m]	X [m]	Y [m]	Z [m]	ΔL [m]
1						0	1.000	-2.600	0.500	0.000
2	90	1.7				0	1.000	-4.300	0.500	1.700
3	0		90	1	☐	0	0.000	-5.300	0.500	3.271
4	0	2.15				0	-2.150	-5.300	0.500	5.421
5	0		-65	1	☐	0	-3.056	-5.877	0.500	6.555

OK Cancel Apply

- For the *Pipe* that connects the *SourceChili* with the *MixerFinal*, we insert a seamless connection. Click *Pipe8* with the right mouse button and select **Segments [context menu] > Edit [segment]**. In the

segments table you can also insert a segment at the beginning of the object which is not possible manually with the mouse.

- Click the *Pipe* and press **F7**.



I..	Tangential angle [°]	Length [m]	Curve angle [°]	Radius [m]	V.	ΔZ [m]	X [m]	Y [m]	Z [m]	ΔL [m]
1						0	11.400	12.000	0.500	0.000
2	59.0362434679265	5.830951894...				0	14.400	7.000	0.500	5.831

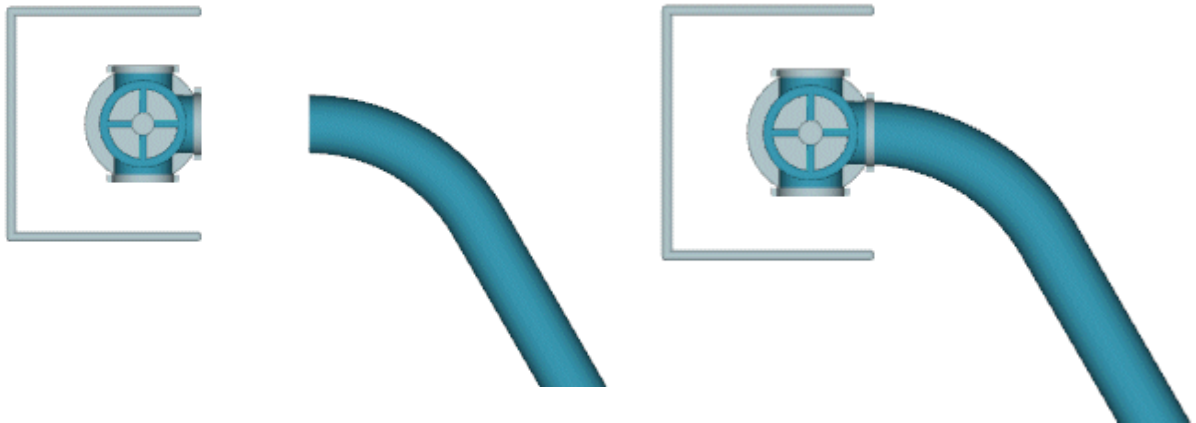
- This opens the **Segments [tab Appearance]** table. Click in row 2 and select **Insert Row**. This inserts an empty row above row 2.

I.	Tangential angle [°]	Length [m]	Curve angle [°]	Radius [m]	Vertical curve	ΔZ [m]	X [m]	Y [m]	Z [m]	ΔL [m]
1						0	10.400	10.000	0.500	0.000
2	0		60	1	☐	0	11.266	9.500	0.500	1.047
3						0	13.866	4.997	0.500	6.247

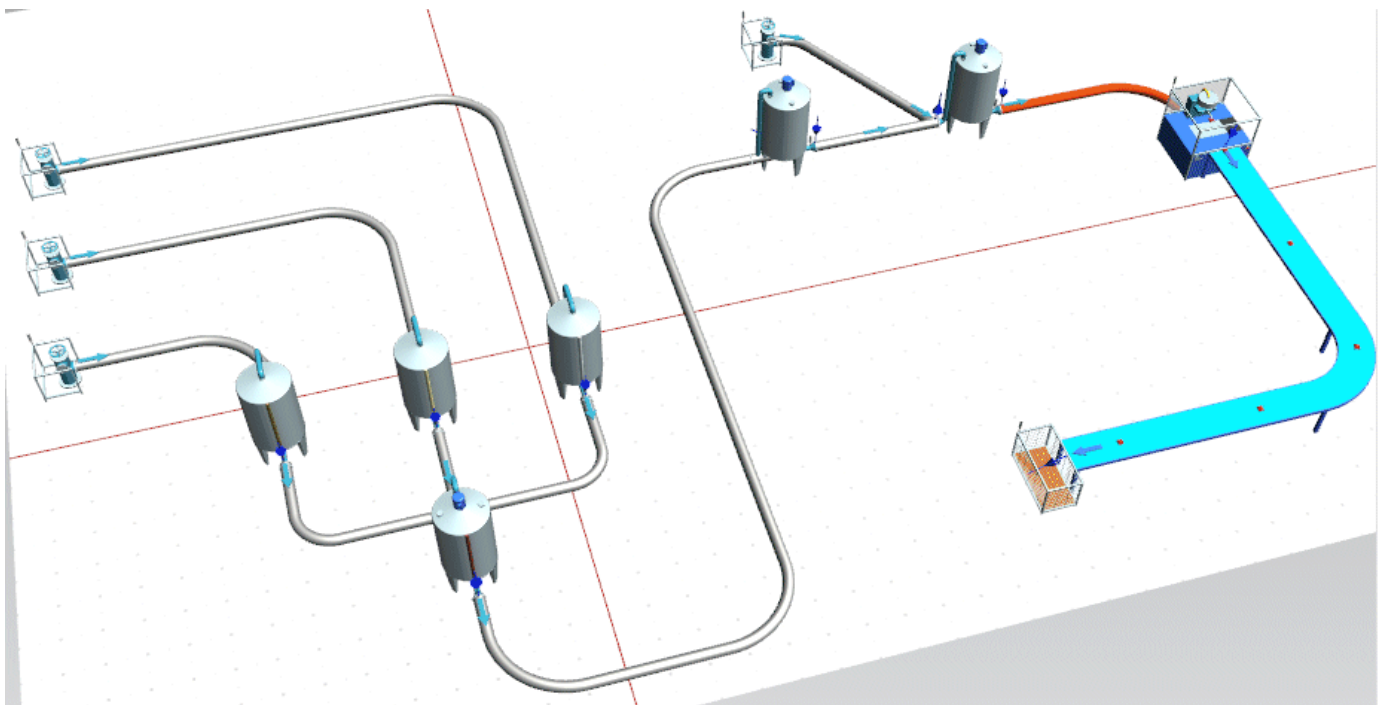
- We typed in these values:

I.	Tangential angle [°]	Length [m]	Curve angle [°]	Radius [m]	Vertical curve	ΔZ [m]	X [m]	Y [m]	Z [m]	ΔL [m]
1						0	10.400	10.000	0.500	0.000
2	0		60	1	☐	0	11.266	9.500	0.500	1.047
3	0	5.2				0	13.866	4.997	0.500	6.247

- The result looks like this. We finally have to adjust the position of the *Pipe*.



The model looks like this with the adjusted *Pipes*.



Compare [Change the Shape of a Segment](#)

Back to [Model the Production of Chocolate Bars](#)

Continuously Mix Fluids

Continuously Mix Juice and Water

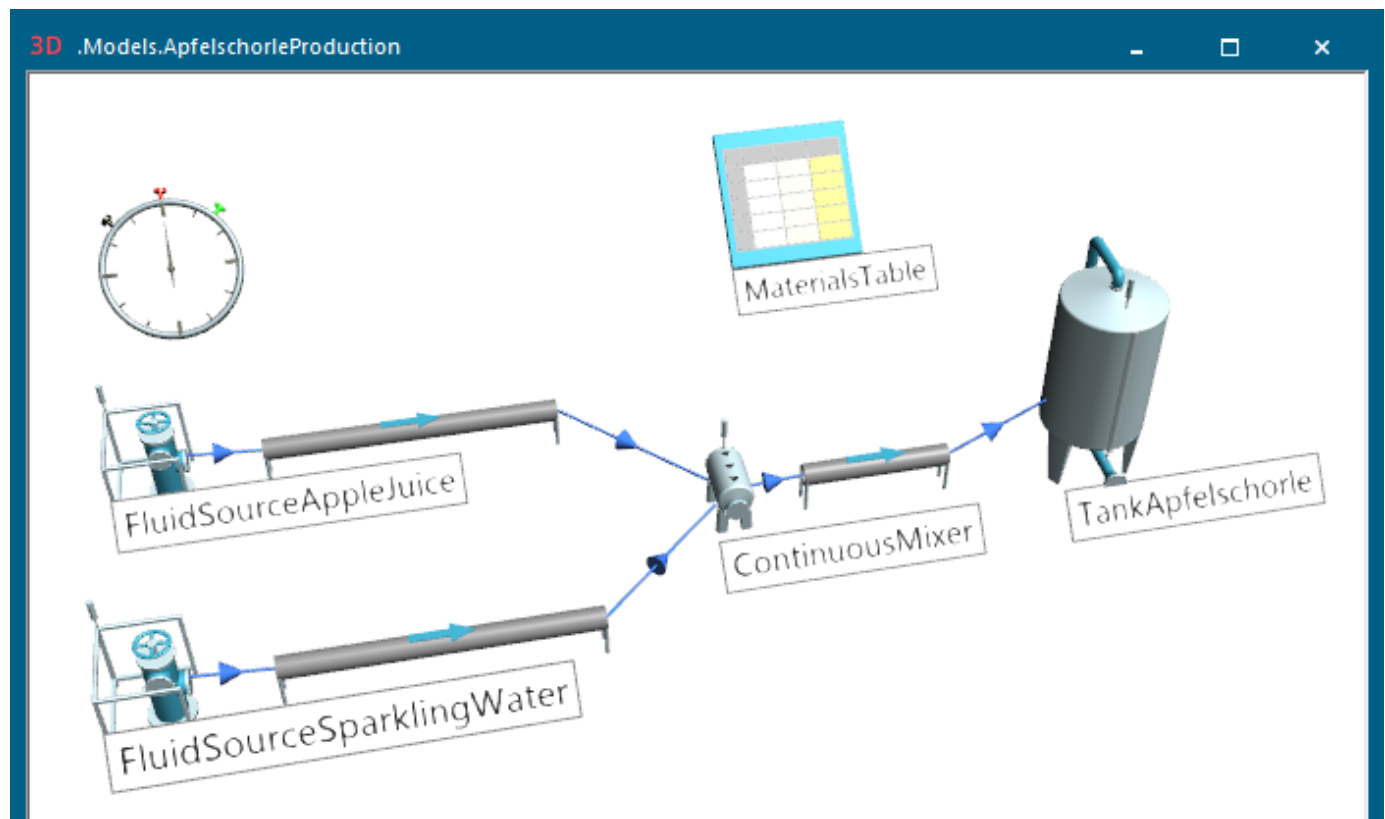
In this example we will model a simple line for producing *Apfelschorle* by mixing *apple juice* and *sparkling water*.

The *FluidSources* → provide the ingredients **apple juice** and **sparkling water**. These then flow through *Pipes* → to the **ContinuousMixer** →, which mixes them. The *Apfelschorle* then flows to the *Tank* →, where it is stored until it will be filled into bottles.

In our example we will demonstrate how to:

- **Configure the Recipe for Mixing the Fluids**
- **Configure the ContinuousMixer and Run the Simulation**

The finished simulation model looks like this:



Back to [Simulate Free-flowing Materials and Fluids](#)

Configure the Recipe for Mixing the Fluids

Before we start building our simulation model, we define the **ingredients** and the end product in the *MaterialsTable*.

The **ingredients** are then going to be used by the *FluidSources* and by the *ContinuousMixer*.

We typed in the **Materials**, the **Product amount**, the **Unit**, **Ingredient 1** and its **Unit**, and **Ingredient 2** and its **Unit**. We also selected the **Colors** of the individual **Materials**.

Our completed **MaterialsTable** looks like this:

SparklingWater											
string 1	real 2	integer 3	real 4	string 5	string 6	real 7	string 8	string 9	real 10	string 11	
string	Material	Density [g/cm³]	Color	Product amount	Unit	Ingredient 1	Amount	Unit	Ingredient 2	Amount	Unit
1	StandardMaterial	1.000	16768582								
2	Apfelschorle	1.000	10551295	2.000	l	AppleJuice	1.000	l	SparklingWater	1.000	l
3	AppleJuice	1.000	5751039								
4	SparklingWater	1.000	16777215								
5											

We then type AppleJuice and SparklingWater into the respective *FluidSources* to make them produce these **Materials**. We did not change any other settings.

.Models.ApfelschorleProduction.FluidSourceAppleJuice

Name: FluidSourceAppleJuice

Label:

Attributes: Outflow Rate: 1 l/s, Material Selection: Constant, Material: AppleJuice, Materials Table: MaterialsTable, Shift Calendar:

Current Material: -, Current Outflow Rate: 0 l/s

.Models.ApfelschorleProduction.FluidSourceSparklingWater

Name: FluidSourceSparklingWater

Label:

Attributes: Outflow Rate: 1 l/s, Material Selection: Constant, Material: SparklingWater, Materials Table: MaterialsTable, Shift Calendar:

Current Material: -, Current Outflow Rate: 0 l/s

Compare [Add Additional Ingredients to the MaterialsTable](#)

Compare [Configure the Recipe in the MaterialsTable](#)

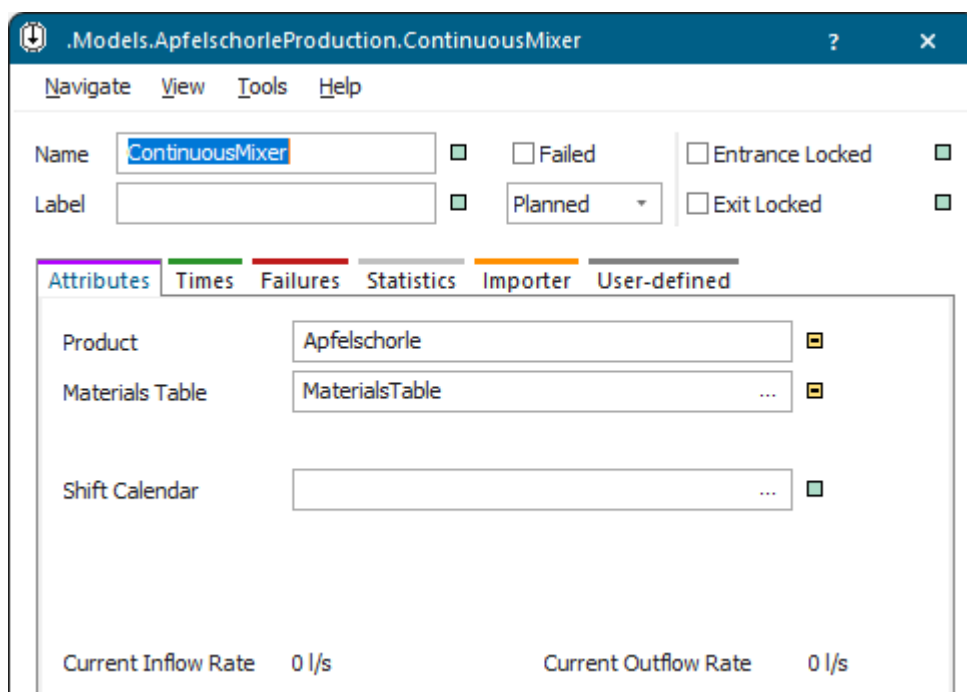
Back to [Continuously Mix Juice and Water](#)

Configure the ContinuousMixer and Run the Simulation

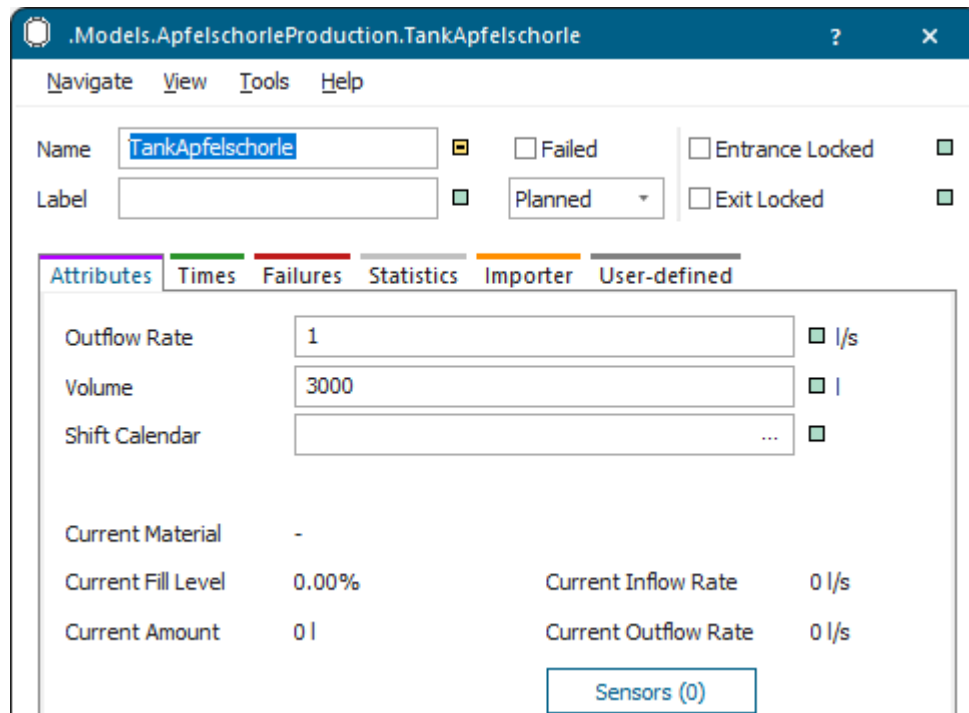
Set the *Product* which the *ContinuousMixer* produces and run the simulation.

After configuring the recipe, we have to tell the *ContinuousMixer* which *Product* it has to create.

To do so, we type `Apfelschorle` into the text box **Materials Table**. We specified the **Apfelschorle** in the *MaterialsTable*.

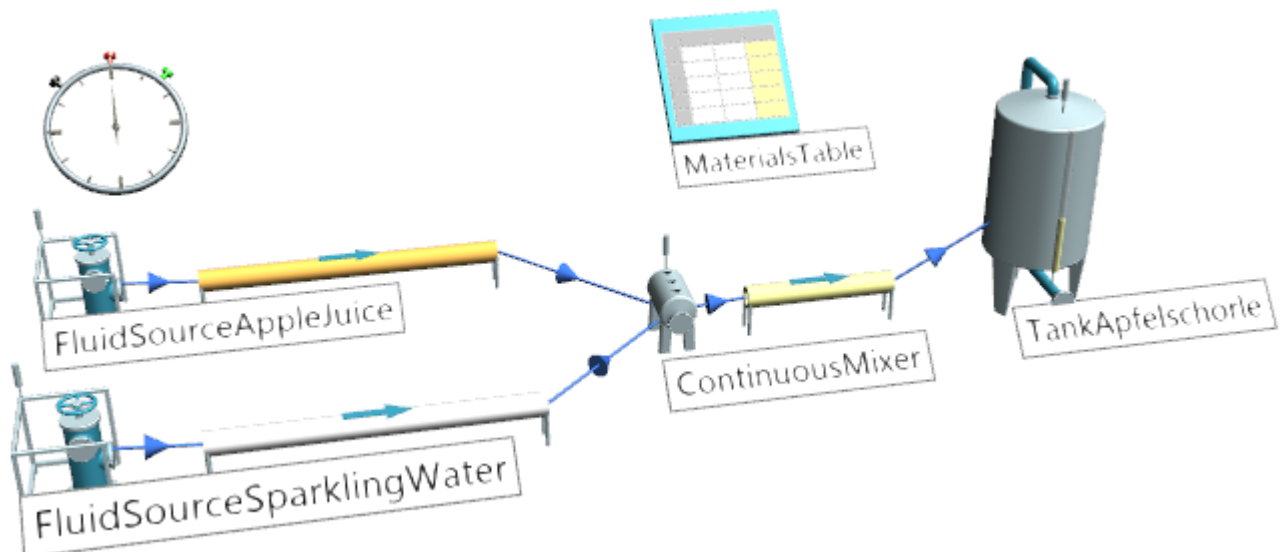


We use the *Tank*, which stores our *Apfelschorle*, with its default settings, namely an **Outflow Rate** of 1 liter per second and a **Volume** of 3000 liters.



We then connect the objects with *Pipes* and *Connectors* and run the simulation.

We can now watch how the *FluidSources* produce the materials with their respective color, which our transparent *Pipes* show, how the *ContinuousMixer* mixes them, and sends the *Apfelschorle* on to the *Tank*. The *Tank* shows its fill level with the indicator at the side of the outflow valve.



Back to [Continuously Mix Juice and Water](#)

Portioning and Deporting Materials

Portion and Deport Materials

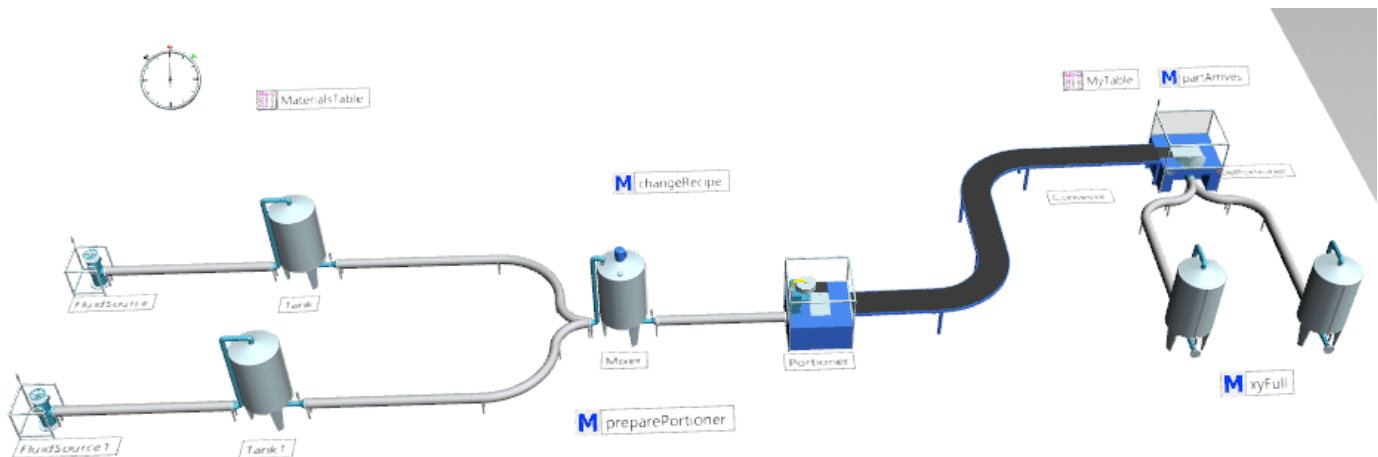
In this example we demonstrate how to mix two different raw materials in the *Mixer* into two products.

The *Mixer* then passes these products on to the *Portioner*, which portions them into *MUs*, and moves them on a *Conveyor* to the *DePortioner*. The *DePortioner* splits the products and lets them flow off into the respective *Tanks*. A tanker truck may then pick them up from there.

The model also shows how to connect the transport of fluids and bulk goods with the normal, continuous material flows, in our case with a *Conveyor*.

Keep in mind that you cannot connect the *Pipes* directly with the fluid objects, but that you have to use *Connectors*.

The finished simulation model looks like this:



In our sample model we will:

- Create the Required Parts
- Configure the Recipe of the Product in the MaterialsTable
- Configure the FluidSources Which Produce the Materials
- Configure the Tanks Which Buffer the Materials
- Configure the Mixer Which Mixes the Materials
- Configure the Portioner Which Portions the Product
- Configure the DePortioner
- Configure the Tanks at the End of the Plant

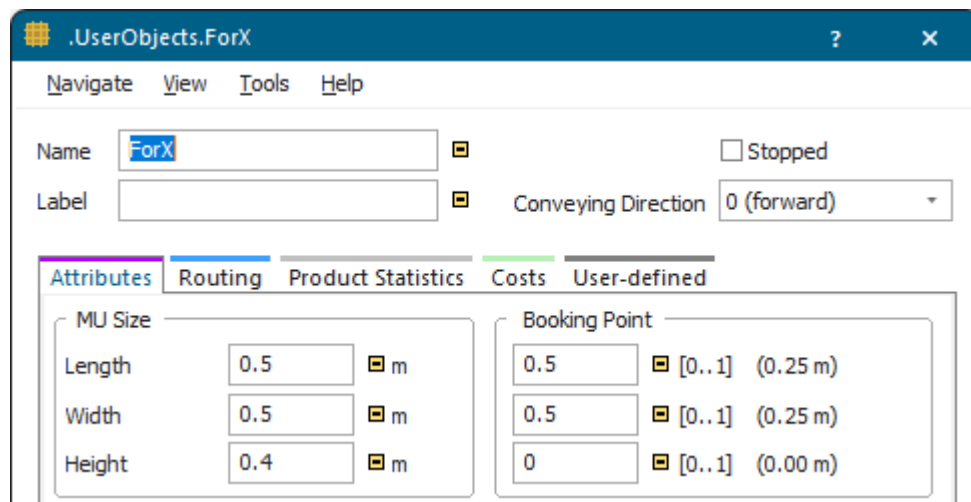
Back to [Simulate Free-flowing Materials and Fluids](#)

Create the Required Parts

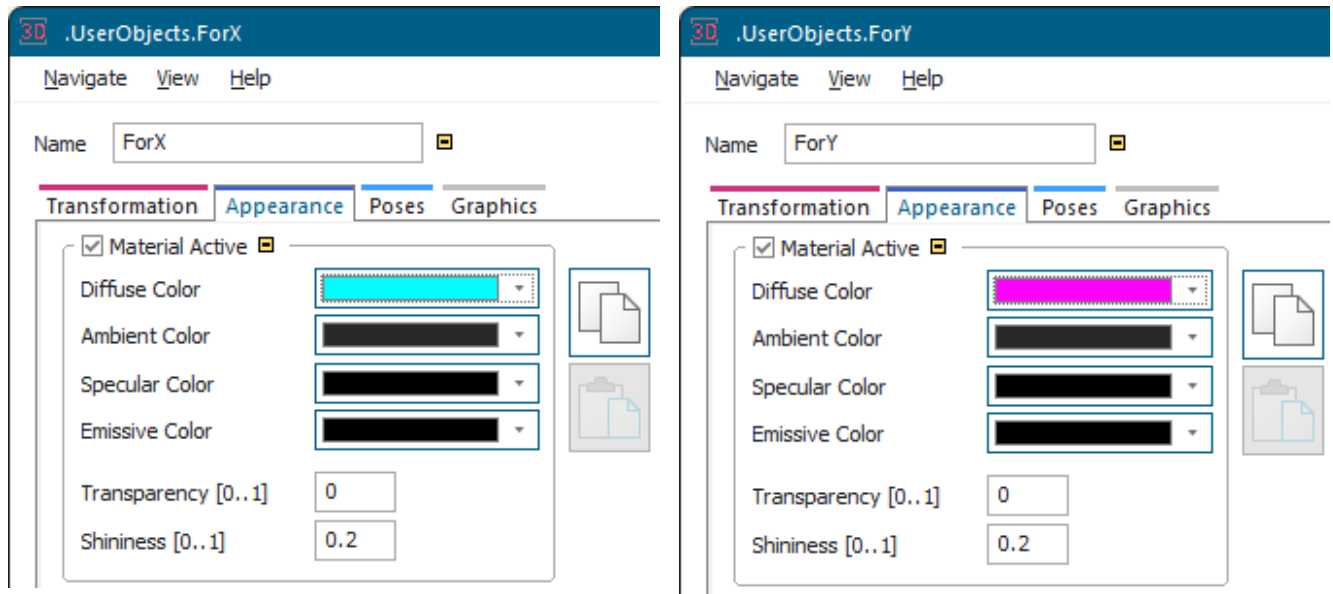
Before we start creating our simulation model, we will create two parts for transporting our bulk goods or our fluid.

Proceed as follows:

- Change to the folder **MUs** in the *Class Library*.
- Click the *Parts* with the right mouse button and select **Duplicate**.
- Repeat this.
- Change to the folder **UserObjects** in which *Plant Simulation* placed the duplicated parts.
- Rename the duplicated *MUs*. We renamed our *MUs* to ForX and ForY and entered these settings for the **MU Size**:



- Change to the tab **Appearance** and select different colors for the *MUs* to be able to distinguish them. We selected the colors below.



- To make 3D scale the MUs, we open their class in the *Class Library* and click the tab **Transformation** in the dialog **Edit 3D Properties**. Then select **Scale Automatically**

The *Portioner* fills the materials, which the *Mixer* mixed, into these MUs.

Back to [Portion and Deportion Materials](#)

Configure the Recipe of the Product in the MaterialsTable

Before we start building our simulation model, we define the **ingredients** and the products in the object *MaterialsTable*.

These are then going to be used by the *FluidSource* , the *Portioner* , and by the *Mixer* .

You can enter the materials and the information pertaining to them in the order in which they occur to you. You are not bound to the order in which the materials are used within the production process. As long as the materials are contained in the **MaterialsTable** , the object, which needs them, will find them.

In most cases you can do that in the *MaterialsTable* in the *class library*, meaning that you do not have to insert an instance of it into the *Frame* that contains your simulation model.

If your simulation model contains a number of *Frames* which require a vast number of different materials, we recommend to insert different *MaterialsTables* into the *Frames*. A single *MaterialsTable* might become too large and unwieldy.

We also entered the **materials**, their **density**, their **color**, the **amount** which we want to produce. We also entered **ingredient 1** and **ingredient 2** and their respective **amount** and **unit**.

As the product amount of the *Mixer* for **Product Y** differs from the product amount in the *MaterialsTable*, *Plant Simulation* adjusts the amounts of the individual ingredients so that the ratio of the ingredients in the recipe is maintained.

Note

When you change the name of materials or of ingredients at a later point in time in the *MaterialsTable*, you also have to manually change this name in the text box **Material** of the *FluidSource* and of the *DePortioner*, and/or the text box **Product** of the *Mixer*. *Plant Simulation* does not automatically do that for you.

Our completed *MaterialsTable* looks like this.

Product X											
	string 1	real 2	integer 3	real 4	string 5	string 6	real 7	string 8	string 9	real 10	string 11
string	Material	Density [g/cm ³]	Color	Product amount	Unit	Ingredient 1	Amount	Unit	Ingredient 2	Amount	Unit
1	Product X	1.000	16768582	6.000	l	Raw material 1	2.000	l	Raw material 2	4.000	l
2	Product Y	1.000	8388863	3.000	l	Raw material 2	2.000	l	Raw material 1	1.000	l
3	Raw material 1	1.000	49152								
4	Raw material 2	1.000	16744448								
5											

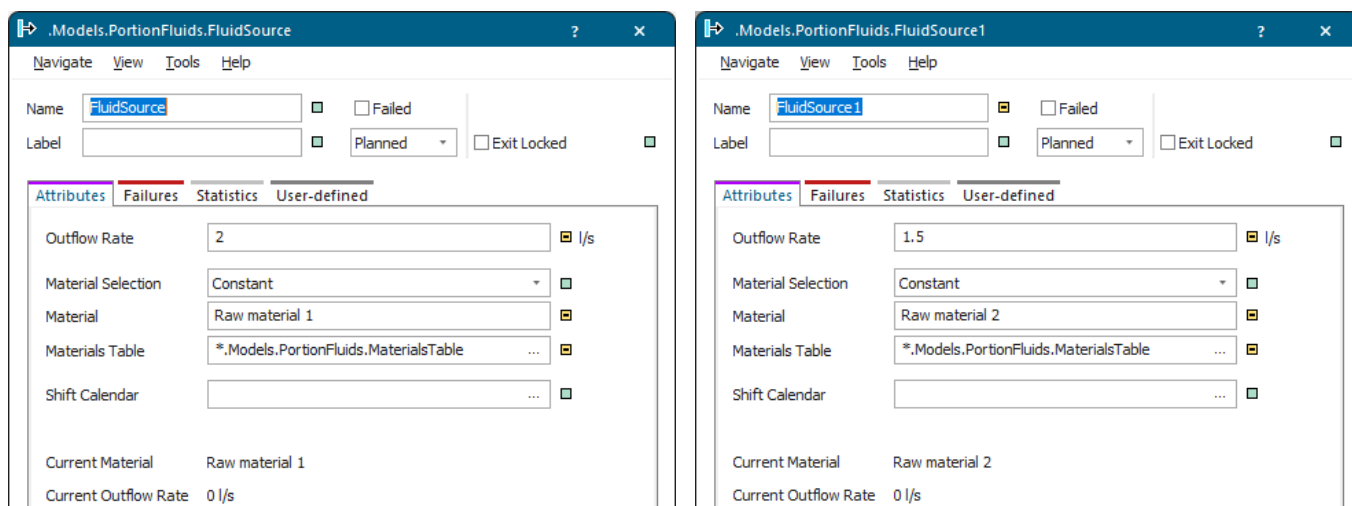
Back to Portion and Deportion Materials

Configure the FluidSources Which Produce the Materials

After we filled in the *MaterialsTable*, we configured the objects of type *FluidSource*, which introduce the raw materials into the facility.

We inserted a **FluidSource** each for **Raw material 1** and **Raw material 2**. We then entered the name of the material into the text box **Material**, i.e., Raw material 1 and Raw material 2. For the *FluidSource* we use an outflow rate of 2 liters per second, for the *FluidSource1* we use an outflow rate of 1.5 liters per second.

Configure the Tanks Which Buffer the Materials

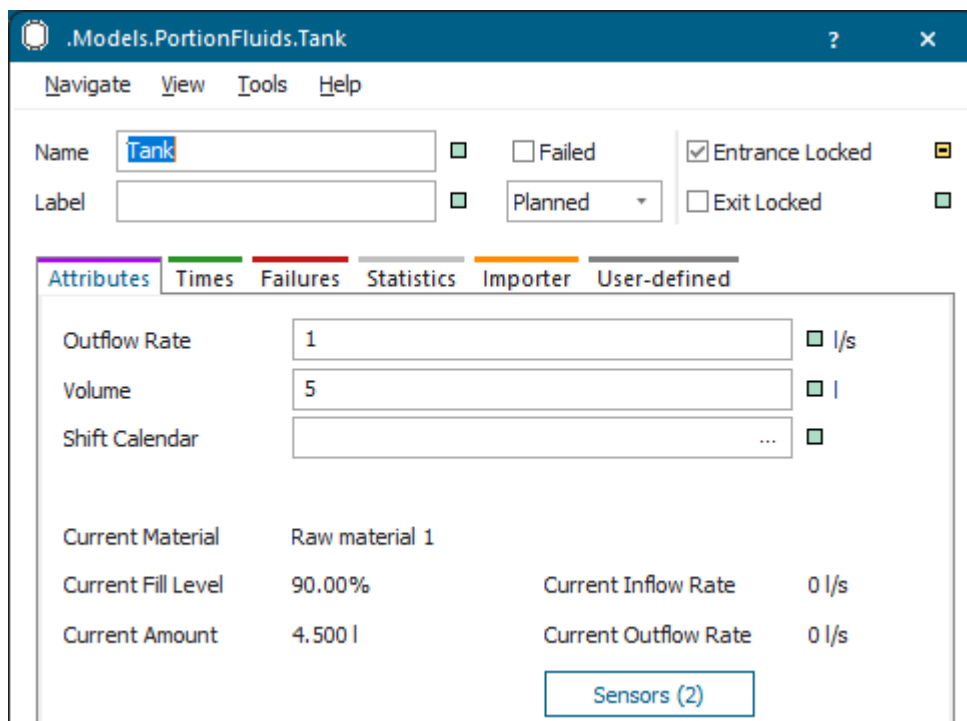


Back to Portion and Deportment Materials

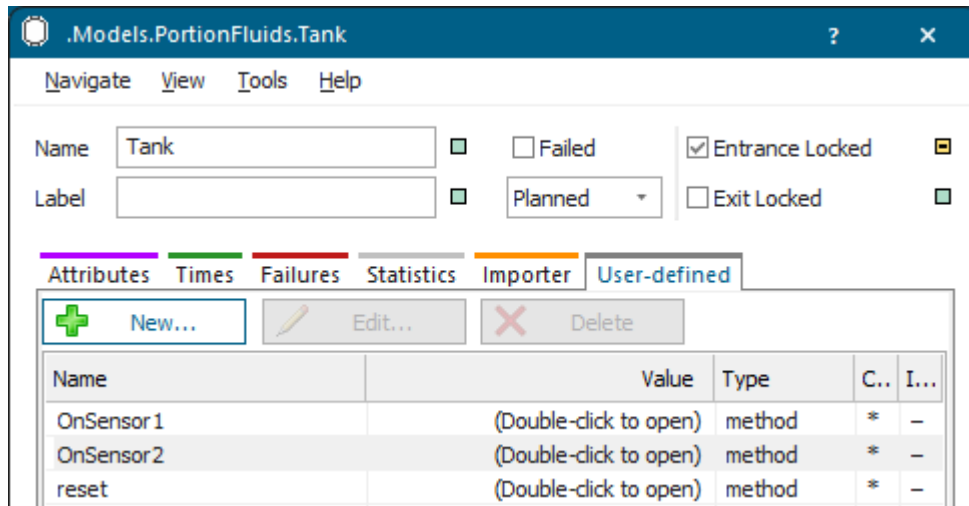
Configure the Tanks Which Buffer the Materials

Next, we inserted an object of type *Tank* each for **Raw material 1** and **Raw material 2**. The *Tanks* store the raw materials temporarily before the *Mixer* mixes them.

We entered the following settings:



- An outflow rate of 1 liter per second into the text box **Outflow rate** of the respective object for each of the materials.
- A volume of 5 liters for each of the materials into the text box **Volume** of the respective object for each of the materials.
- Two **sensors** and their **controls** with the settings shown below.



We then programmed the **Sensor Controls** and **Reset Control**:

- For the **Sensor Control 1** we typed this instruction:

```
param SensorID: integer, Exceeded: boolean
self.~.EntranceLocked := false
```

- For the **Sensor Control 2** we typed this instruction:

```
param SensorID: integer, Exceeded: boolean
self.~.EntranceLocked := true
```

- For the **Reset Control** we typed this instruction:

```
self.~.EntranceLocked := false
```

Repeat these steps for the second **Tank [object]**.

Next, we inserted *Pipes* with the default settings between the *FluidSources* and the respective *Tanks* and we connected the two *Pipes* with the *FluidSources* and the *Tanks* with *Connectors*.

Note

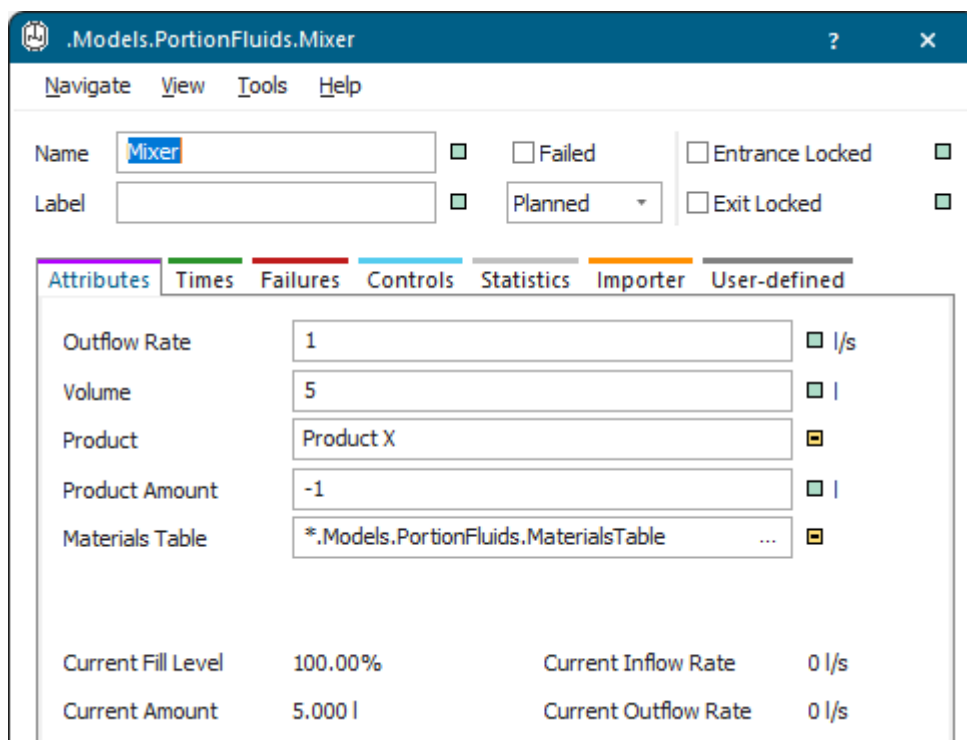
You have to connect the individual fluid objects with *Connectors*. You cannot connect them with *Pipes* only.

Back to Portion and Deportion Materials**Configure the Mixer Which Mixes the Materials**

In the next step we inserted a *Mixer*, which transforms the raw materials and the products.

We entered the following settings:

- An **Outflow Rate** of 1 liter per second.
- A **Volume** of 6 liters.
- The name of the **Product**.



.Models.PortionFluids.Mixer

Navigate View Tools Help

Name: ☒ ☐ Failed ☐ Entrance Locked ☒

Label: ☒ Planned ☐ Exit Locked ☒

Attributes Times Failures Controls Statistics Importer User-defined

Outflow Rate: ☒ l/s

Volume: ☒ l

Product: ☒

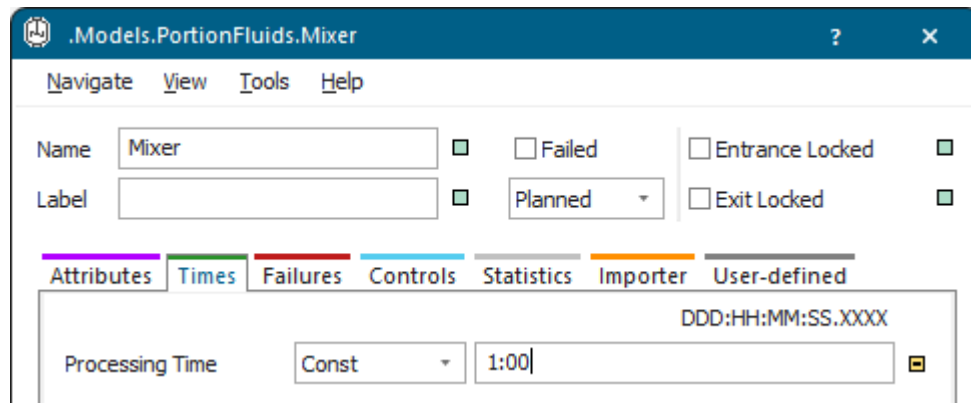
Product Amount: ☒ l

Materials Table: ... ☒

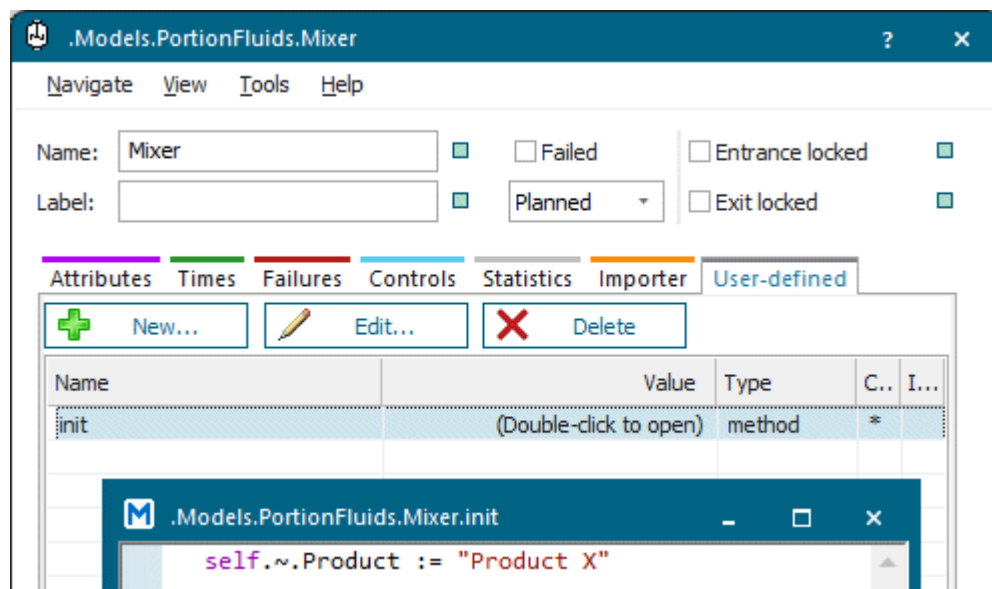
Current Fill Level: 100.00% Current Inflow Rate: 0 l/s

Current Amount: 5.000 l Current Outflow Rate: 0 l/s

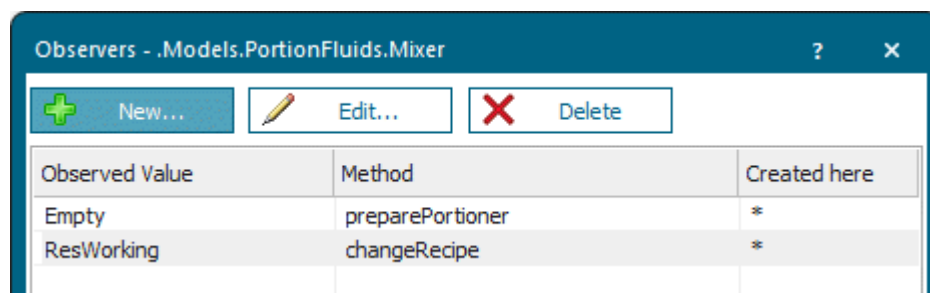
- A **Processing Time** on the tab **Times**.



- A **Init Control** on the tab **User-defined**. The **Init Control** makes sure that the first product mixed in the **Mixer** always is the same.



- We also created an **observer** for the *read-only attributes* named *ResWorking* and *Empty* and the corresponding controls so that the *Mixer* changes the recipe of the material.



```
// observer for ResWorking
param Attribute: string, previousValue: any
if NOT ?.ResWorking AND NOT ?.Failed AND NOT ?.Stopped
  if ?.Product = "Product X"
    ?.Product := "Product Y"
```

```
else
    ?.Product := "Product X"
end
end
```

```
// observer for Empty
param Attribute: string, previousValue: any
if ?.Leer
    if ?.Product = "Product Y"
        Portioner.MUPath := .UserObjects.ForY
        Portioner.AmountPerMU := 3
    else
        Portioner.MUPath := .UserObjects.ForX
        Portioner.AmountPerMU := 2
    end
end
end
```

The observer for *ResWorking* changes the product for the following processing step of the *Mixer* at the end of the processing time. The product can only be changed once the *Mixer* is empty. At this point in time, which the observer for *Empty* recognizes, the *Portioner* can be prepared for the new product.

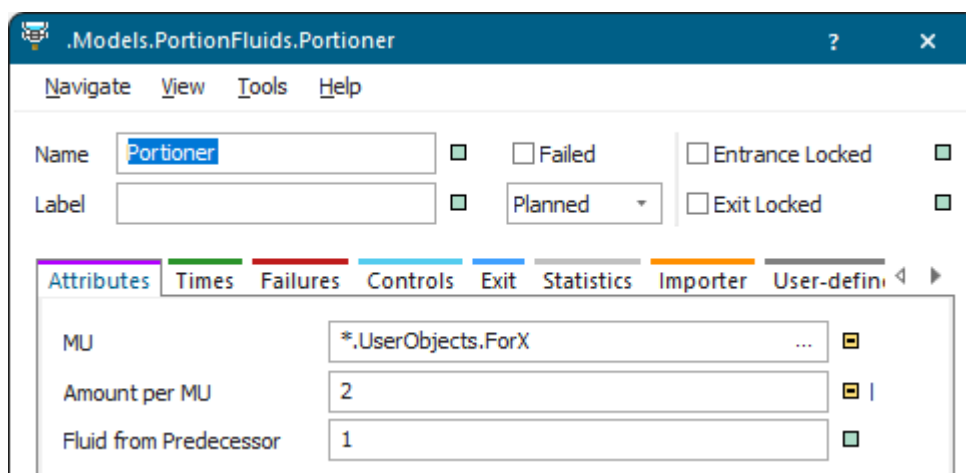
We finally connected the *Mixer* and the *Portioner* with a *Pipe* with the default settings and with *Connectors*.

Back to [Portion and Deportion Materials](#)

Configure the Portioner Which Portions the Product

We now configure the *Portioner*, which splits the material into portions and, for example, pours it into containers.

- We selected the **MU**, which the **Portioner**  is to create from the products. We selected the *MU* named *ForY*, which we created above.
- We entered the **Amount per MU** of the product. In our example we fill 2 liters into a *MU* respectively.



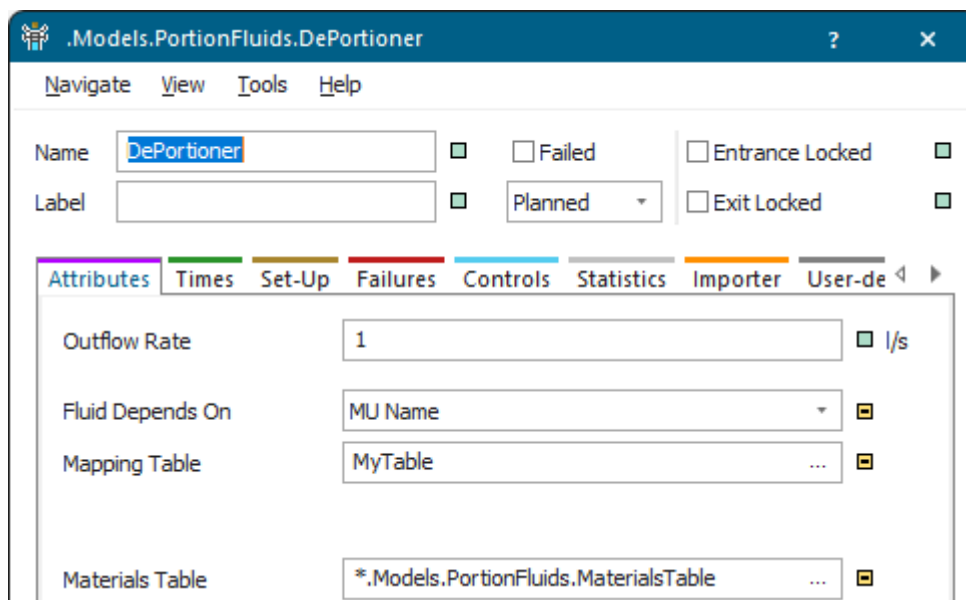
Back to Portion and Deportment Materials

Configure the DePortioner

Insert a *Conveyor* between the *Portioner* and *DePortioner*. The *Conveyor* transports the portioned materials in their containers. We used the default settings of the *Conveyor*.

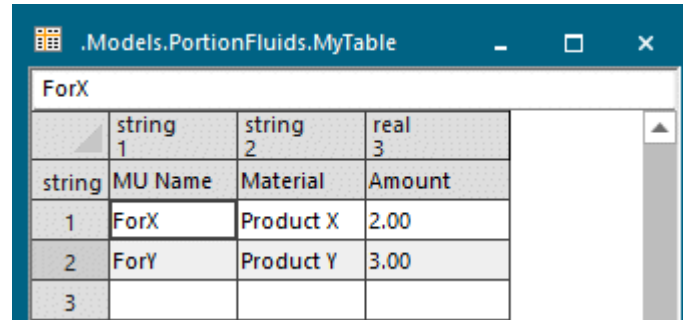
Configure the **DePortioner** .

- For the **outflow rate** we used the default setting of 1 liter per second.



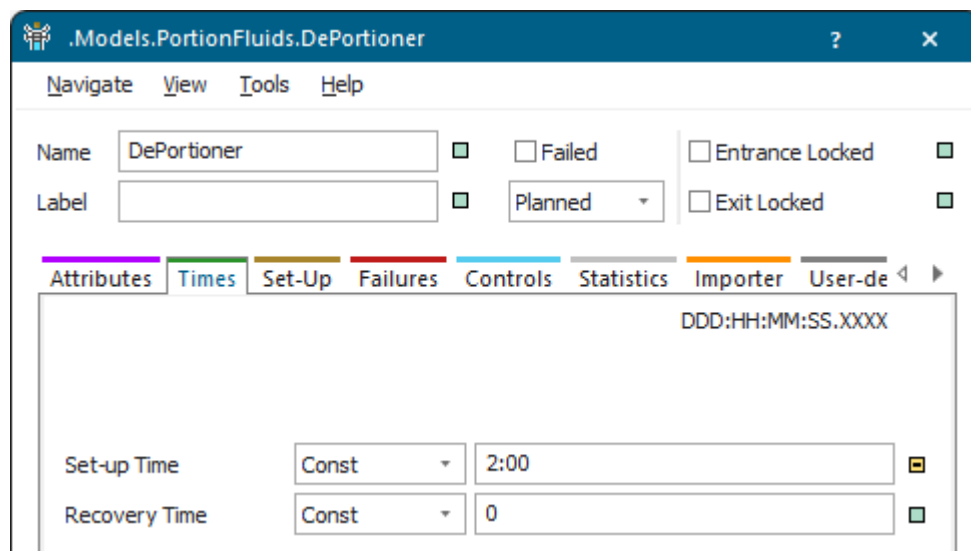
- We selected **Fluid Depends On > MU Name** so that the *DePortioner* creates the material and the amount of the fluid depending on the **MU names**, which are defined in the **Mapping Table**.

- We selected the *DataTable* named *MyTable* as our **mapping table**. We entered the **MU name**, the **Material**, and the **Amount** of the product, which we want to portion.



	string 1	string 2	real 3
	MU Name	Material	Amount
1	ForX	Product X	2.00
2	ForY	Product Y	3.00
3			

- We entered a **Set-up Time** of 2 minutes to set the *DePortioner* up for the new product when the product changes.

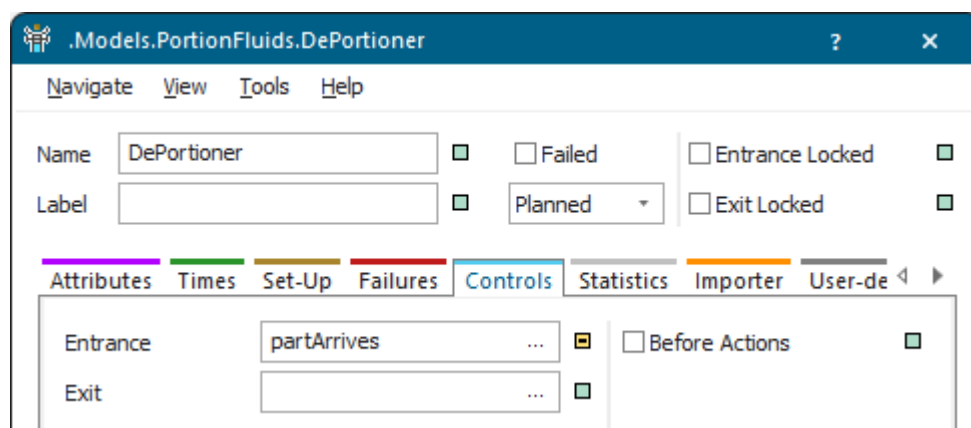


Name: DePortioner ☐ Failed ☐ Entrance Locked ☐
 Label: ☐ Planned ☐ Exit Locked ☐

Attributes Times Set-Up Failures Controls Statistics Importer User-de

Set-up Time: Const 2:00 ☐
 Recovery Time: Const 0 ☐

- The **Entrance Control** named *partArrives* opens or closes the *Pipes* to the individual *Tanks* into which the products flow.



Name: DePortioner ☐ Failed ☐ Entrance Locked ☐
 Label: ☐ Planned ☐ Exit Locked ☐

Attributes Times Set-Up Failures Controls Statistics Importer User-de

Entrance: partArrives ☐ Before Actions ☐
 Exit: ☐

We typed this source code into the method *partArrives*:

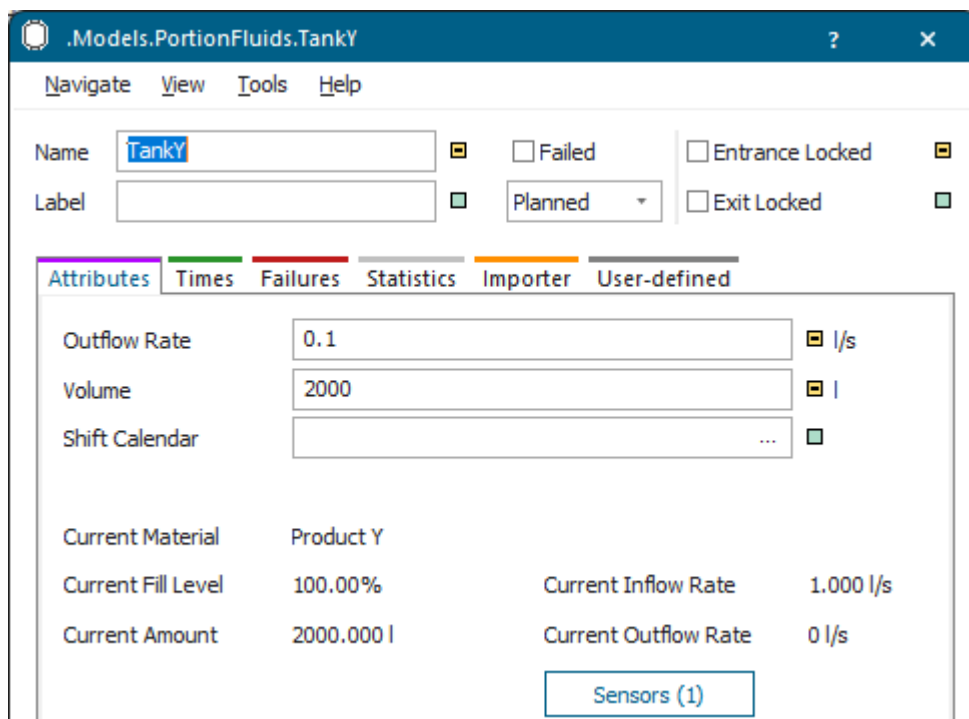
```
// entrance control of the DePortioner
if @.MaterialName = "Product X"
    PipeX.PipeOpened := true
    PipeY.PipeOpened := false
    @.MaterialName := "Product Y"
else
    PipeX.PipeOpened := false
    PipeY.PipeOpened := true
end
```

Insert the *Pipes* which connect the *DePortioner* with the succeeding *Tanks*. Connect the objects with *Connectors*.

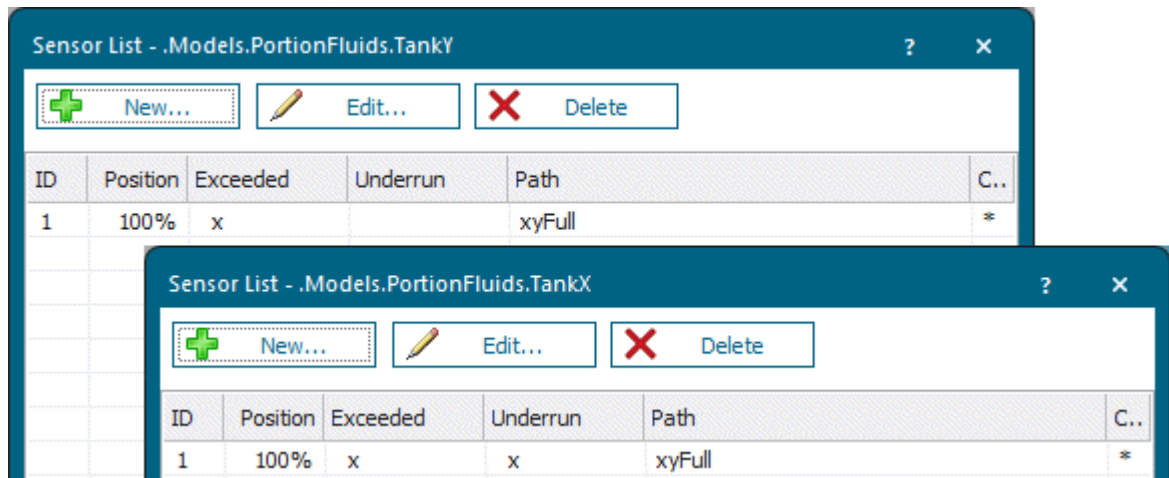
Back to Portion and Deportment Materials

Configure the Tanks at the End of the Plant

In our last step we configure the two *Tanks* at the end of the production facility, which store the products. We entered an **outflow rate** of 0.1 liter per second and a volume of 2000 liters for both *Tanks*.



In addition we created a **sensor** in the **Tank** each with the following settings:

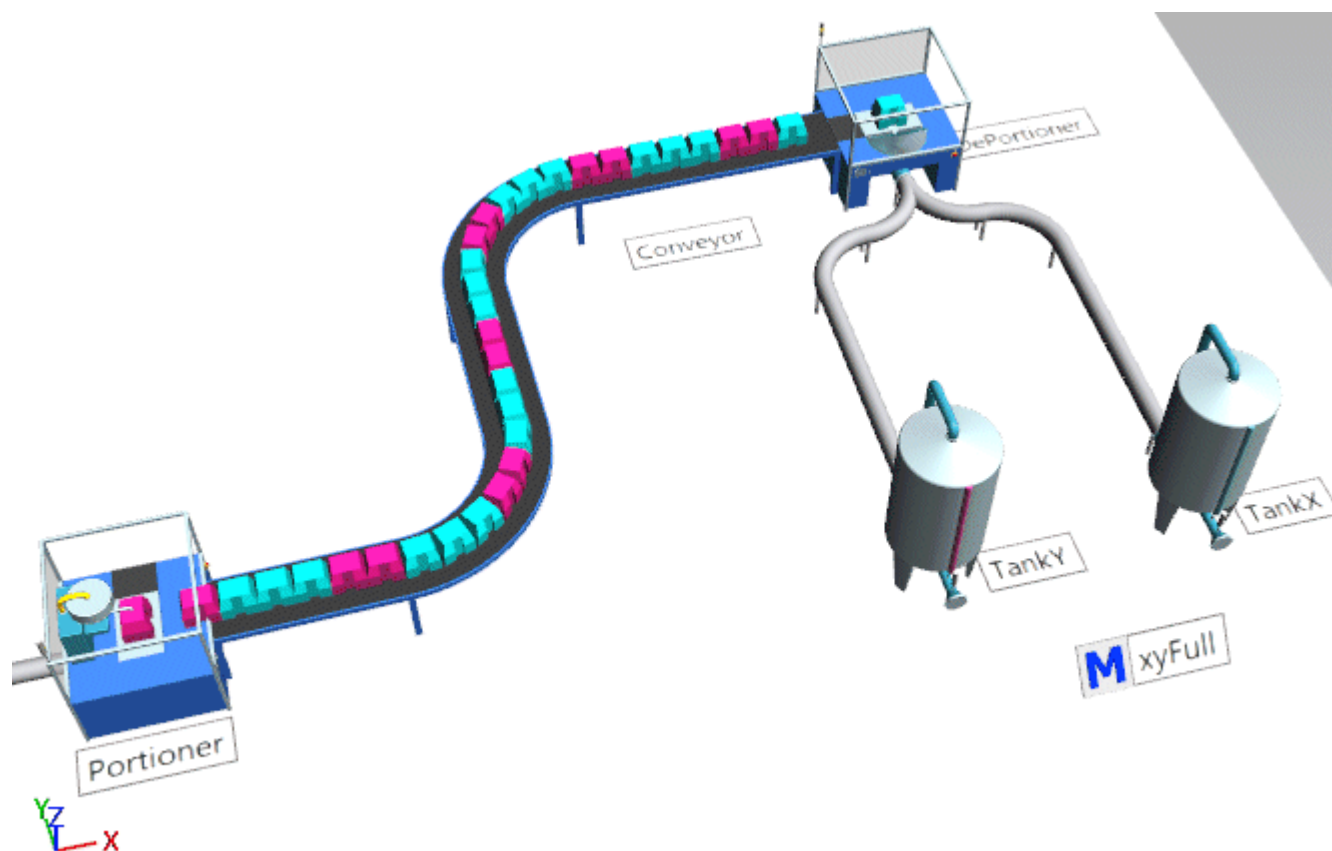


We typed in this source code into the sensor control named *xyFull*.

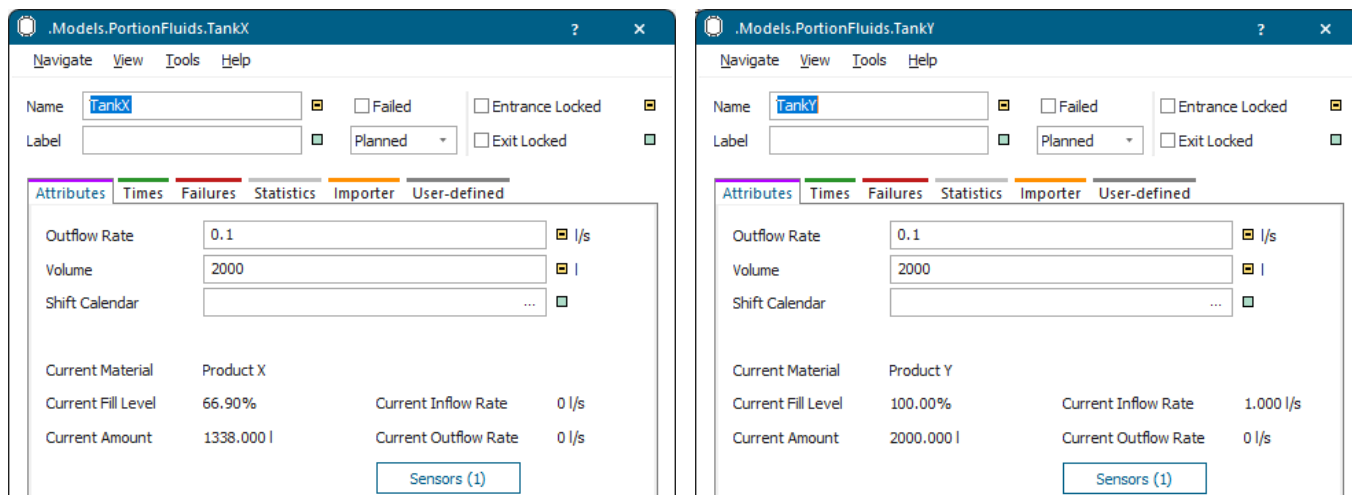
```
// sensor control of TankX and TankY
param SensorID: integer, Exceeded: boolean
EventController.stop(true)
```

We finally created a **user-defined attribute** with the name *reset* for both *Tanks* and typed in this source code:

```
self.~.EntranceLocked := false
```



When we run the die simulation, we can watch the *Mixer* mixing two products from the two raw materials, which the *Portioner* then splits up in portions. The *Conveyor* moves these filled portions to the *DePortioner*, which dissolves the portions and then lets them flow off to the designated *Tanks*. As soon as one of the *Tanks* has reached its defined volume of **2000** liters, the simulation stops.



Back to [Materials](#)

Producing Fluids in a Fixed Sequence

Produce Fluids in a Fixed Sequence

In this example we model a simple production line in which a *FluidSource* produces three materials in a fixed sequence.

The *FluidSource*  produces the materials in the sequence and with the amount in the order in which you enter them into the **sequence table**.

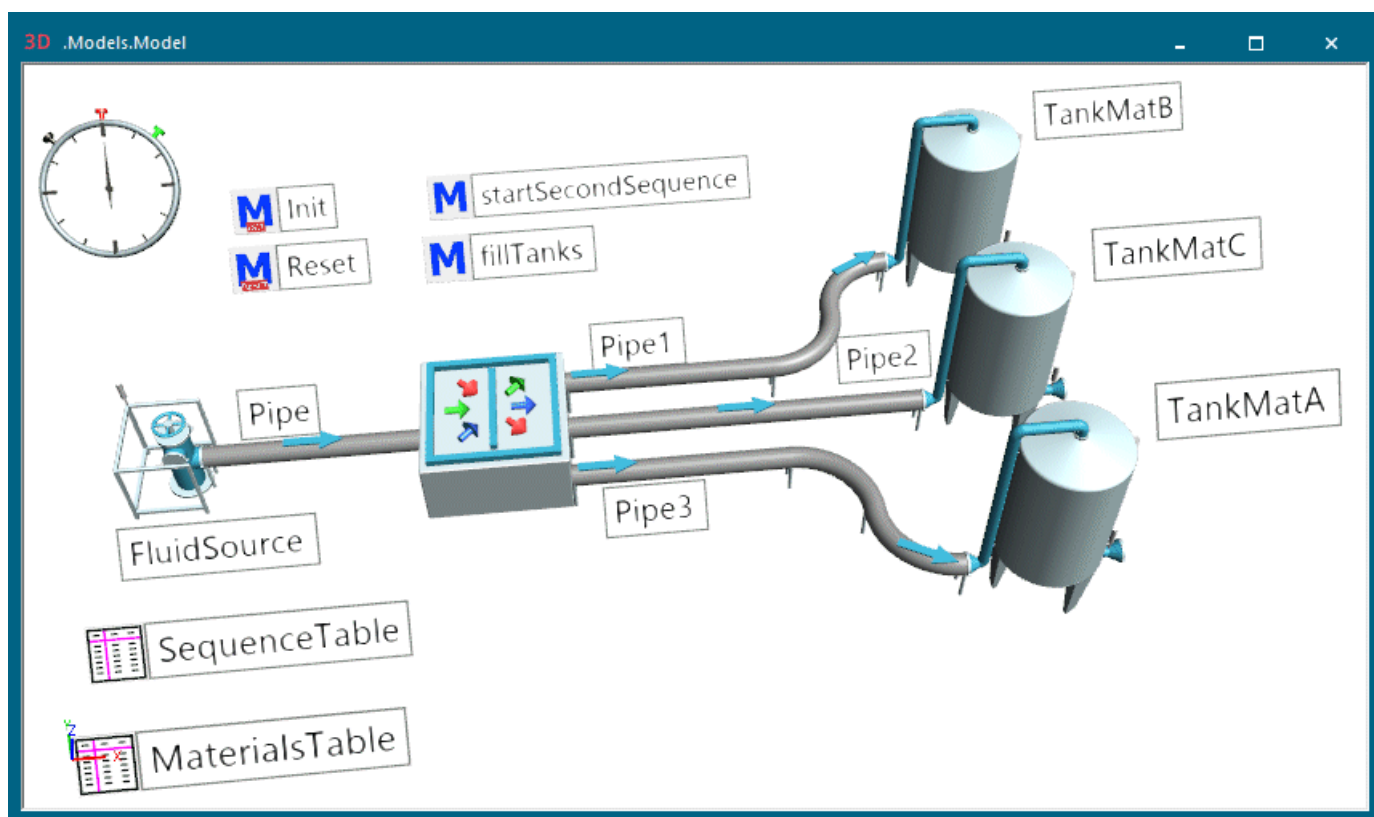
The materials then flow through *Pipes* to a *PatchMatrix* , which distributes the materials via three *Pipes* to the *Tanks*  storing the individual materials. We define how the *PatchMatrix* fills the *Tanks* in a *Method*, which is called by the *init method*.

In our example we will demonstrate how to:

- **Configure FluidSource, MaterialsTable, and SequenceTable**
- **Configure the Tanks Receiving the Materials**
- **Define How the PatchMatrix Fills the Tanks**
- **Run the Simulation to View the Results**

We also inserted a *Pipe* between the *FluidSource* and the *PatchMatrix* and three *Pipes* from the *PatchMatrix* to the three *Tanks*. Remember that you have to establish connections between *Pipes* and *fluid objects* with *Connectors*.

The finished simulation model looks like this:



Back to [Simulate Free-flowing Materials and Fluids](#)

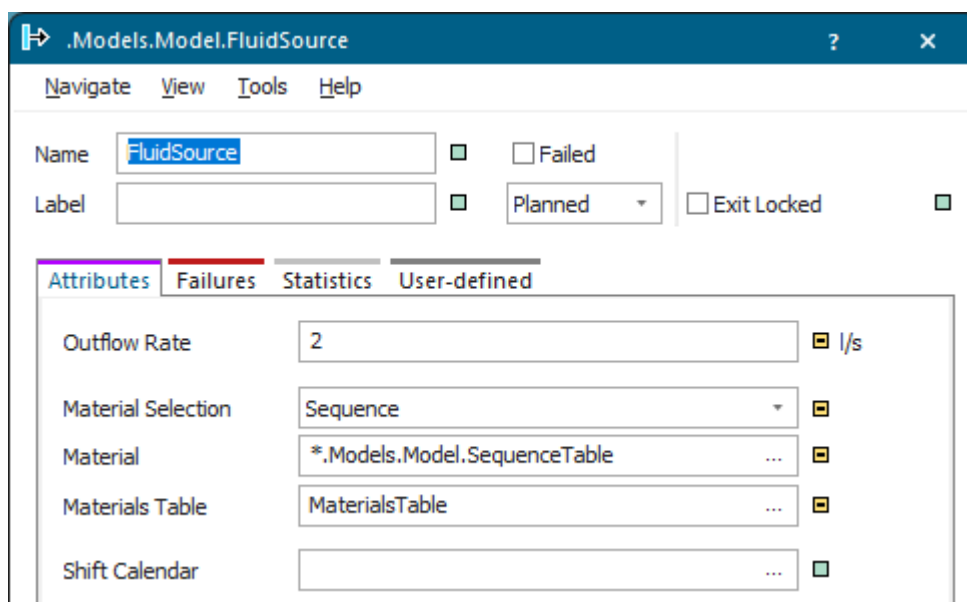
Configure FluidSource, MaterialsTable, and SequenceTable

After we inserted the objects into our simulation model, we first configure the *FluidSource* that produces the fluid materials.

- As we want it to produce them in a fixed sequence, we select **Material Selection > Sequence** on the tab **Attributes**.

The *FluidSource* produces the materials in the sequence and with the amount in the order in which you enter them into the **Sequence Table**.

The setting **Sequence** processes the sequence only once, not repeatedly. When the sequence is processed once, the *FluidSource* does not produce any additional materials.



- We then entered the materials MatA, MatB, and MatC into the *MaterialsTable*. To be able to distinguish them when they flow through the *Pipes*, we assigned them different colors.

	string 1	real 2	integer 3	real 4	string 5
	Material	Density [g/cm³]	Color	Product amount	Unit
1	MatA	1.000	16711935		
2	MatB	1.000	65354		
3	MatC	1.000	12591104		

- We finally entered the **sequence** in which the *FluidSource* produces the materials and the required amount of the **Material** into the *SequenceTable*. The *FluidSource* first produces 1000 liters of MatB, then 1500 liters of MatC, and finally 800 liters of MatA.

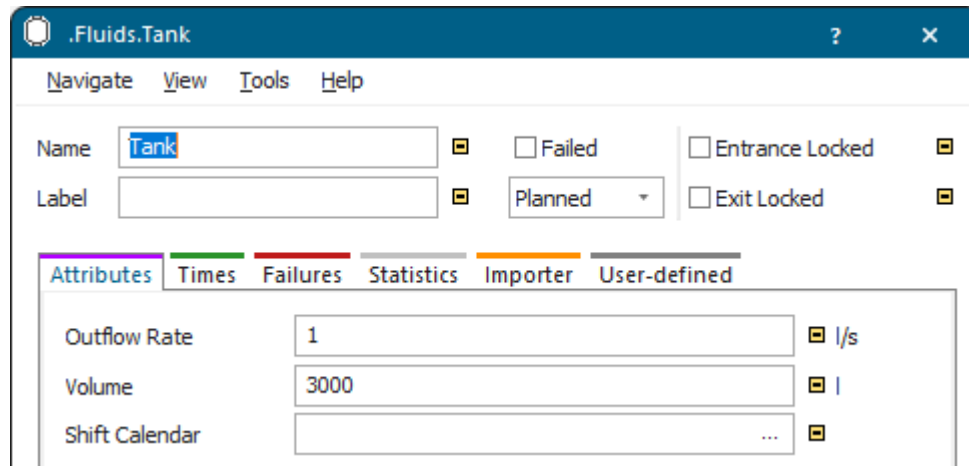
	string 1	real 2
	Material	Amount
1	MatB	1000.000
2	MatC	1500.000
3	MatA	800.000

Back to [Simulate Free-flowing Materials and Fluids](#)

Configure the Tanks Receiving the Materials

Next we inserted a *Tank* each for the three materials.

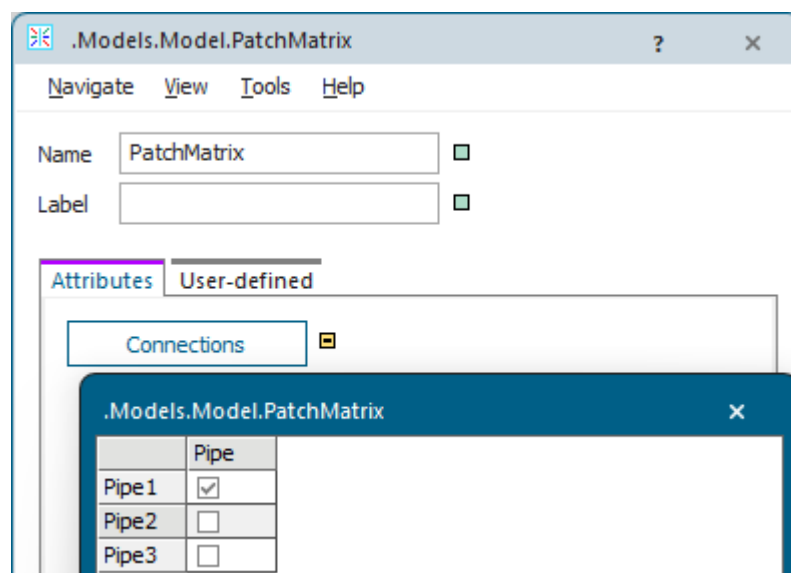
As we use the same settings for each of them, namely an **Outflow rate** of 1 liter per second and a **Volume** of 3000 liters, we enter them into the class of the *Tank*:



Back to Simulate Free-flowing Materials and Fluids

Define How the PatchMatrix Fills the Tanks

When we connect the *PatchMatrix* with the incoming and outgoing *Pipes*, it enters them into the **Connections** table.



Normally you set which outgoing *Pipes* the *PatchMatrix* connects with the incoming *Pipes* by selecting the respective check box(es).

Our modeling situation is different though. Our *FluidSource* produces different amounts of the three materials in a **fixed sequence**. The *PatchMatrix* thus has to connect the incoming *Pipe* with the respective outgoing *Pipe* to fill the three *Tanks* with the correct material.

We programmed that in our method named `fillTanks`.

```
var currMaterial = ""

waituntil FluidSource.CurrentMaterial /= currMaterial
currMaterial = FluidSource.CurrentMaterial
PatchMatrix.setConnections(1, 1, true)    // lets the first material flow
into the outgoing Pipe1

waituntil FluidSource.CurrentMaterial /= currMaterial
currMaterial = FluidSource.CurrentMaterial
PatchMatrix.setConnections(1, 1, false)   // closes Pipe1
PatchMatrix.setConnections(1, 2, true)    // lets the second material flow
into the outgoing Pipe2

waituntil FluidSource.CurrentMaterial /= currMaterial
currMaterial = FluidSource.CurrentMaterial
PatchMatrix.setConnections(1, 2, false)   // closes Pipe2
PatchMatrix.setConnections(1, 3, true)    // lets the third material flow
into the outgoing Pipe3
```

The method `fillTanks` is called in our *init method*.

```
PatchMatrix.resetConnections
fillTanks
```

We also reset the connections of the *PatchMatrix* in our *reset method*.

```
PatchMatrix.resetConnections
```

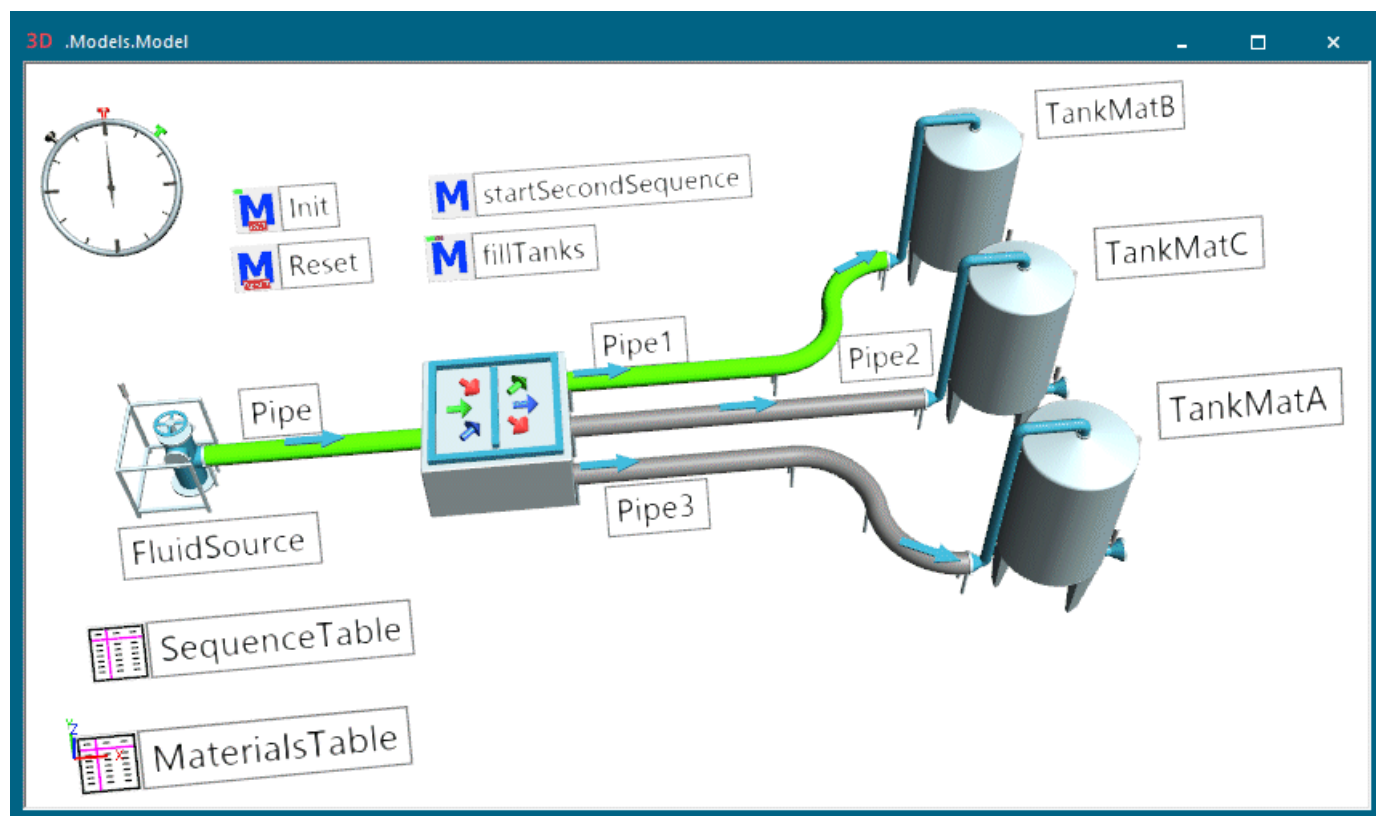
Back to [Simulate Free-flowing Materials and Fluids](#)

Run the Simulation to View the Results

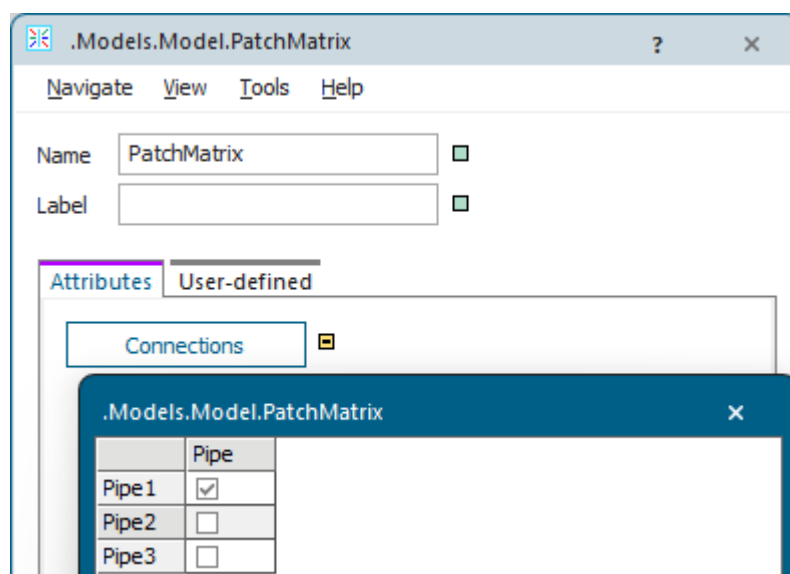
When we run the simulation, we can watch the *FluidSource* producing the materials in the sequence specified in the *SequenceTable* one after the other. The color of the *Pipe* shows which material flows. The *PatchMatrix* then sends the arriving material on to the correct *Tank* by opening the respective *Pipe* leading to that *Tank*.

To better see the results, we set a real-time factor of 42 in the *EventController*.

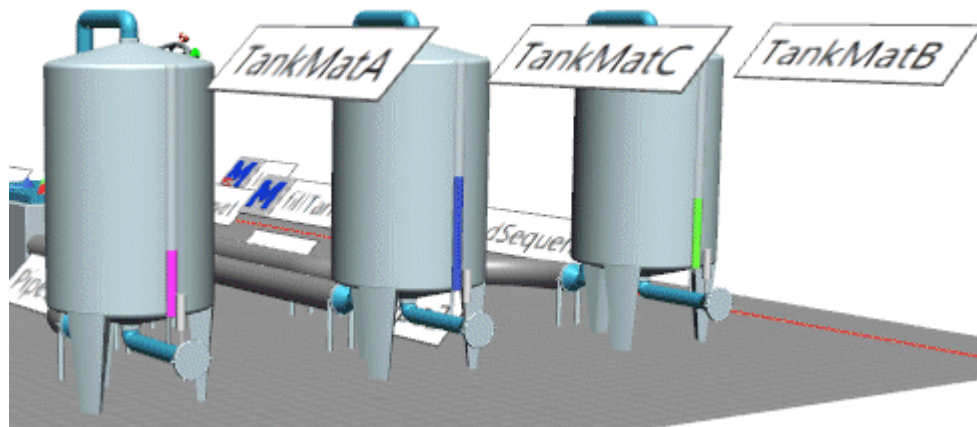
First the *FluidSource* produces 1000 liters of *MatB*, designated by the green color we specified in the *MaterialsTable*, and passes the material on to the *PatchMatrix*.



It opens *Pipe1* to let *MatB* flow into the *TankMatB*.



Next flows *MatC* and finally *MatA*. At the end of the simulation all *Tanks* are filled with the specified amount of the specified material.



If we need to run the same sequence of materials again, with a pause in between, we can program a *Method* to do so. We named our *Method* *startSecondSequence* and entered this source code.

```
print EventController.SimTime  
  
FluidSource.Material = void  
FluidSource.Material = SequenceTable  
  
fillTanks
```

To execute it during our simulation run, we entered this instruction into our *init method*:

```
&startSecondSequence.executeIn(1:00:00)  
PatchMatrix.resetConnections  
  
fillTanks
```

At the end of the simulation run the *Tanks* consequently contain twice the amount of the materials.

